The Power of Low Rank: Fast CKKS Functional Bootstrapping for High-Precision Lookup Tables

Zhihao Li[1], Xuan Shen[2,3], Cheng Hong[⋆1], Ruida Wang[2,3], Xianhui Lu[⋆⋆2,3], and Tao Wei[1]

[1] Ant Group
[2] State Key Laboratory of Cyberspace Security Defense, Institute of Information Engineering, CAS
[3] School of Cyber Security, University of Chinese Academy of Sciences

Abstract. The CKKS fully homomorphic encryption scheme has traditionally been viewed as suitable only for approximate arithmetic. However, recent work (Alexandru et al., Crypto 2025) has introduced functional bootstrapping techniques that enable accurate lookup tables (LUTs) evaluation in CKKS. Nevertheless, to deal with the high precision problem, the state-of-the-art scheme (Dumezy et al., TCHES 2026) requires reshaping the LUTs into a matrix, which incurs $O(P)$ multiplications for a size-P table and dominates runtime.

We first observe that LUT matrices for many practical functions are often highly structured, with exact or numerical rank much smaller than matrix dimension. We then develop a spectral framework for the LUT evaluation problem, which characterizes the relationship between function classes and the singular value decomposition. This framework yields exact rank bounds for structured function classes such as separable functions, and establishes exponential decay of singular values (implying low numerical rank) for smooth analytic functions. Building on this framework, we propose Low Rank Multiplexer Tree Functional Bootstrapping (LRMT-FBT), which evaluates the LUT via the singular values and singular vectors instead of direct matrix multiplication. This reduces the homomorphic multiplication cost from $O(P)$ to $O(r\sqrt{P})$, where r denotes the rank of matrix, while also supporting extensions to multi-value and multi-input settings.

We implement LRMT-FBT in OpenFHE and evaluate it across different spectral classes. We also introduce implementation optimizations to improve the bootstrapping efficiency. At high precision, LRMT-FBT provides substantial performance improvements for common low rank functions compared with Dumezy et al. Typically, for $P = 2^{20}$, our method accelerates the LUT evaluation step by 196.9× for Step ($r = 1$) and 99.6× for ReLU ($r = 3$), yielding functional bootstrapping speedups of 5.3× and 5.1×, respectively.

Keywords: CKKS, homomorphic encryption, functional bootstrapping, lookup tables, Low Rank Multiplexer Tree.

⋆ Corresponding author: vince.hc@antgroup.com
⋆⋆ Corresponding author: luxianhui@iie.ac.cn

1 Introduction

Fully homomorphic encryption (FHE) enables arbitrary computation on encrypted data, but natively supports only addition and multiplication [27,10,26,14,19]. Nonlinear operations such as comparisons are central to privacy-preserving inference and encrypted database queries, yet they must be expressed through these two operations alone. A common approach is polynomial approximation [15,33], which represents the target function by a finite-degree polynomial and therefore incurs an intrinsic approximation error. This error is negligible for smooth functions under high-degree polynomial approximation, but for discontinuous functions such as sign and comparison it cannot be eliminated by any finite degree. Lookup tables (LUTs) avoid this dependence, since they represent the function by its values on all P inputs without any smoothness characteristics. However, homomorphic LUT evaluation still accumulates noise, so deep circuits must bootstrap to refresh the ciphertext [27]. Functional bootstrapping (FBT) combines LUT evaluation and noise refresh in a single bootstrapping step, which reduces the overall cost.

The TFHE [24] and FHEW schemes [17,19] are the first to support functional bootstrapping, with subsequent work extending the supported precision [29,6,7]. However, batch capability remains challenging in these schemes, where amortized variants [31,22,30] still fall short of the throughput required to evaluate thousands of function values in parallel, as is common in neural-network inference. In contrast, the CKKS scheme [14] is well suited to high-throughput settings, as its SIMD mechanism evaluates all packed slots in a single bootstrapping invocation [2,3,9,13]. Specifically, Alexandru et al. [2] propose the first general CKKS-based FBT (AKP-FBT) through trigonometric Hermite interpolation. However, the interpolation degree grows linearly with the table size, which makes the method impractical once the input precision exceeds roughly 14 bits [25]. Bian et al. [8] reduce the degree required for smooth functions, but the same linear growth remains for general discrete LUTs. Dumezy et al. [25] overcome this scaling limitation by introducing the collapsed multiplexer-tree based bootstrapping (CMT-FBT), which evaluates LUTs of up to 2^{20} entries at a multiplicative depth independent of the table size.

CMT-FBT proceeds in three steps. First, the encrypted input $x \in \mathbb{Z}_P$ is decomposed into its radix-p digits, and a low-precision FBT maps each digit to the Kronecker-delta vector of its value. Second, the Kronecker product aggregates these per-digit vectors into the row and column selectors $\boldsymbol{\delta}_{\mathrm{MSD}}, \boldsymbol{\delta}_{\mathrm{LSD}} \in \{0,1\}^{\sqrt{P}}$. Third, the LUT is reshaped into a $\sqrt{P} \times \sqrt{P}$ matrix $\mathbf{M}_f$, and the output $f(x)$ is computed by the dense matrix evaluation $\boldsymbol{\delta}_{\mathrm{MSD}}^{\top} \mathbf{M}_f \, \boldsymbol{\delta}_{\mathrm{LSD}}$. The dense matrix evaluation requires $O(P)$ homomorphic multiplications and is the dominant cost of CMT-FBT at high precision. We observe that the computational overhead of CMT-FBT is independent of the function, since this method does not depend on the entries of $\mathbf{M}_f$. The LUT matrices of practical activation functions are far from generic, and their rank is typically much smaller than the matrix dimension.

Table 1. Spectral classification of LUTs at $P = 2^{20}$ and the resulting acceleration of LRMT-FBTover CMT-FBT. The rank r is the exact algebraic rank for Class A and the numerical rank at threshold $\varepsilon = 1/(2P)$ for Class B. In Class C, LRMT-FBTdegrades to CMT-FBT.

Function	Spectral class	Application	r	Trunc. error	LUT speedup	Bootstrapping speedup
$\mathrm{sign}(x)$	A (step)	comparison	1	0 (exact)	196.9×	5.3×
$\mathrm{Step}(x-c)$	A (step)	classification	1	0 (exact)	196.9×	5.3×
$\exp(x)$	A (separable)	exponential	1	0 (exact)	196.9×	5.3×
$x \bmod k$	A (periodic)	modular reduction	1	0 (exact)	196.9×	5.3×
$\lfloor x/k \rfloor$	A (piecewise-const)	quantization	1	0 (exact)	196.9×	5.3×
$\mathrm{ReLU}(x), \lvert x \rvert$	A (piecewise-affine)	NN activation	2	0 (exact)	99.7×	5.1×
$\sin(x), \cos(x)$	A (separable)	trigonometric	2	0 (exact)	99.7×	5.1×
x^2	A (monomial)	polynomial kernel	3	0 (exact)	66.7×	5.1×
$\mathrm{GeLU}(x)$	B (smooth)	transformer	5	5.2×10^{-8}	40.5×	4.9×
$\mathrm{SiLU}(x)$	B (smooth)	transformer	5	1.5×10^{-8}	40.5×	4.9×
$\mathrm{Sigmoid}(x)$	B (smooth)	NN activation	5	4.3×10^{-9}	40.5×	4.9×
$\tanh(x)$	B (smooth)	NN activation	5	2.0×10^{-7}	40.5×	4.9×
Uniform random LUT	C (flat)	worst case	1024	—	1.0×	1.0×

1.1 Contributions

Building on this observation, we substitute the dense matrix evaluation in CMT-FBT with low-rank factorization based on the singular value decomposition (SVD), yielding LRMT-FBT (Low-Rank Multiplexer-Tree FBT). Our main contributions are summarized as follows.

1. **A Spectral Framework for LUT Matrices.** We develop a spectral framework that characterizes the correspondence between the LUT matrix $\mathbf{M}_f$ and the singular value decomposition. This correspondence determines the minimum rank of $\mathbf{M}_f$ through the error reduction analysis. Specifically, we first prove that the error of a low-rank LUT approximation is solely determined by the singular value at index $r+1$ of $\mathbf{M}_f$. Building on this, we establish two independent singular value decay mechanisms, i.e., algebraic separability and analytical smoothness, and use them to classify LUTs into the following three categories:
   - *Class A (exactly low-rank).* Functions in class A admit an algebraic decomposition on the index grid, covering piecewise-affine maps (step, sign, ReLU, $|x|$) and separable functions (exp, sin, cos), without approximation error during evaluation.
   - *Class B (numerically low-rank).* For smooth activations in Class B, such as Sigmoid, tanh, GeLU, and SiLU, we show that the singular values exhibit exponential decay, so that a rank of order $O(\log P)$ suffices for the CKKS correctness threshold.
   - *Class C (spectrally flat).* For non-structured functions (such as uniform random LUT), there are no effective low-rank approximations of their singular values across the entire spectrum.

   We show the ranks of some common functions in Table 1, where classification remains valid under the CKKS scheme.

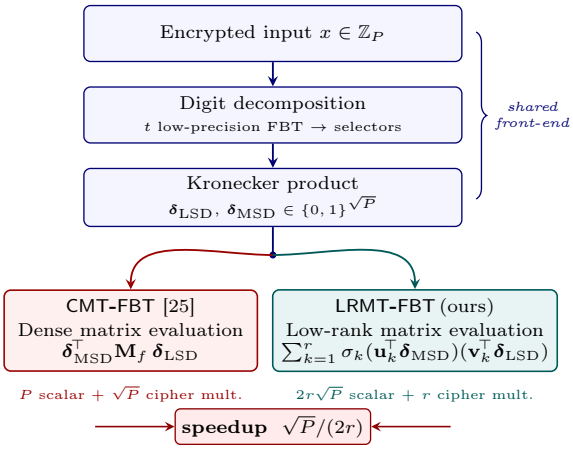


Fig. 1. Pipeline of CMT-FBT and LRMT-FBT for single-input LUT evaluation at precision P.

2. **Low-rank multiplexer-tree bootstrapping.** The LUT evaluation step of CMT-FBT is its dominant cost at high precision, requiring P scalar and $\sqrt{P}$ ciphertext multiplications. We propose a new low-rank functional bootstrapping that factorizes $\mathbf{M}_f$ through its singular value decomposition and replaces the dense matrix evaluation with a rank-r separable expansion as

$$\boldsymbol{\delta}_{\mathrm{MSD}}^{\top}\mathbf{M}_f\,\boldsymbol{\delta}_{\mathrm{LSD}} \approx \sum_{k=1}^{r}\sigma_k(\mathbf{u}_k^{\top}\boldsymbol{\delta}_{\mathrm{MSD}})(\mathbf{v}_k^{\top}\boldsymbol{\delta}_{\mathrm{LSD}}).$$

   This separable expansion enables the evaluation of the LUT using r separable rank-one terms instead of a dense matrix, reducing the number of scalar multiplications to $2r\sqrt{P}$ and ciphertext multiplications to r. In addition, by combining it with digit decomposition and Kronecker product steps, we introduce a new low-rank multiplexer-tree functional bootstrapping (LRMT-FBT) as shown in Figure 1. Finally, we also extend it to more advanced variants, such as the multi-value, multi-input, and slot-independent bootstrapping.

3. **Implementation optimization and evaluation.** We implement LRMT-FBT in the OpenFHE library and evaluate it across all three spectral classes at precisions up to $P = 2^{20}$. We further introduce the lazy conjugation, lazy relinearization, and the rescale technique to reduce the concrete cost of the bootstrapping. Experimental results at $P = 2^{20}$ show that LRMT-FBT achieves a 196.9× speedup in LUT evaluation and a 5.1× speedup in overall bootstrapping for functions of rank 2 (such as ReLU). Finally, we also show the multi-threaded performance and memory results.

1.2 Related Work

Functional bootstrapping has been instantiated in CKKS, BFV/BGV, and TFHE/FHEW schemes, each with distinct trade-offs in latency, throughput, and arithmetic capabilities. We focus on CKKS scheme, as its SIMD arithmetic is particularly suitable for the amortized evaluation of high-precision LUTs.

FBT in CKKS. The CKKS scheme was originally designed for approximate arithmetic [14]. Subsequent works integrate discrete computation into CKKS via error cleaning techniques [23], while Chung et al. [20] develop leveled LUT evaluation and Bae et al. [3,4] propose discrete CKKS bootstrapping for bits and small integers. Alexandru et al. [2] formulate a general single-input CKKS functional bootstrapping framework that combines LUT evaluation and noise cleaning in a single step, supporting arbitrary single-input LUTs. Dumezy et al. [25] combine digit decomposition with a multiplexer tree to extend CKKS LUT evaluation to larger tables and higher-precision settings. Existing CKKS-based methods do not exploit the inherent low-rank structure of high-precision lookup tables, motivating the low-rank functional bootstrapping proposed in this work.

FBT in BFV/BGV. BFV and BGV natively support exact arithmetic over finite fields and utilize SIMD packing for amortized evaluation. Liu and Wang [34] develop LUT evaluation for BFV/BGV bootstrapping via finite-field interpolation, and their later relaxed functional bootstrapping construction [38] further reduces the cost by restricting correctness to a designated subset of the plaintext space. Nevertheless, this line of work incurs substantially higher bootstrapping overhead than CKKS schemes. Moreover, the coupling between the plaintext modulus and the slot structure limits its scalability to higher-precision settings.

FBT in TFHE/FHEW. FHEW and TFHE schemes introduced functional bootstrapping via blind rotation, enabling low-latency LUT evaluation on a single ciphertext [24,19,39,21]. Subsequent work expanded the supported LUT sizes through tree-based bootstrapping [29] and circuit bootstrapping [18,37]. Furthermore, amortized TFHE bootstrapping has also been explored [31,30], but the amortized performance remains inferior to that of CKKS schemes. Consequently, FHEW and TFHE schemes are less suitable for the high-throughput, high-precision setting considered in this work.

1.3 Paper organization

Section 2 reviews the CKKS scheme and the CMT-FBT framework. Section 3 shows the spectral structure of LUT matrices and establishes rank bounds for each class. Section 4 presents the LRMT-FBT algorithm and its noise analysis and correctness proof. Section 5 outlines our implementation and experimental evaluation. Section 6 concludes the paper.

2 Preliminaries

2.1 Notation

All logarithms are base 2 unless explicitly mentioned. Vectors are denoted in bold lowercase (e.g., $\mathbf{u}$), and matrices in bold uppercase (e.g., $\mathbf{M}$). For an integer $q > 1$, $\mathbb{Z}_q$ denotes the residue ring $\mathbb{Z}/q\mathbb{Z}$. For a power-of-two integer N, let $\mathcal{R} = \mathbb{Z}[X]/(X^N + 1)$ and $\mathcal{R}_q = \mathcal{R}/q\mathcal{R} = \mathbb{Z}_q[X]/(X^N + 1)$. A polynomial $a(X) = \sum_{i=0}^{N-1} a_i X^i \in \mathcal{R}$ (or $\mathcal{R}_q$) is represented by the coefficient vector $\mathbf{a} = (a_0, \ldots, a_{N-1})^\top$. For vectors $\mathbf{u}, \mathbf{v} \in \mathbb{R}^m$, $\langle \mathbf{u}, \mathbf{v} \rangle = \mathbf{u}^\top \mathbf{v}$ denotes their inner product, and $\mathbf{u} \otimes \mathbf{v} \in \mathbb{R}^{mn}$ their Kronecker product. For a matrix $\mathbf{M} \in \mathbb{R}^{m \times n}$, $\|\mathbf{M}\|_{\max} = \max_{i,j} |M_{i,j}|$ is the entrywise max-norm, $\|\mathbf{M}\|_2 := \max_{\|x\|_2 = 1} \|\mathbf{M}x\|_2$ the spectral norm, and $\|\mathbf{M}\|_F$ the Frobenius norm. For an arbitrary function $f : \mathbb{Z}_P \to \mathbb{R}$, we associate the P-entry lookup table $\mathrm{LUT}(f) = (f(0), f(1), \ldots, f(P-1))^\top \in \mathbb{R}^P$. We denote by $\mathbf{e}_i$ the i-th standard basis vector, whose i-th entry is one and whose remaining entries are zero. Table 2 presents the symbols used throughout the paper.

2.2 CKKS Scheme

CKKS [14] is an RLWE-based homomorphic encryption scheme for approximate arithmetic over real and complex numbers. We work with its RNS variant [12] in this paper. Let $L+1$ distinct primes $q'_0, \ldots, q'_L$ satisfy $q'_i \equiv 1 \pmod{2N}$, and set $Q'_\ell = \prod_{i=0}^{\ell} q'_i$ for $\ell \in \{0, \ldots, L\}$, where L is the maximum multiplicative depth. Let Δ be a power of two with $\Delta < q'_0$, denoting the scaling factor. Write $M = 2N$, $\zeta = \exp(2\pi \mathrm{i}/M)$, and $\mathbb{Z}_M^* = \{x \in \mathbb{Z}_M \mid \gcd(x, M) = 1\}$. The canonical embedding $\sigma : \mathcal{R} \to \mathbb{C}^N$ is defined by $\sigma(a) = (a(\zeta^j))_{j \in \mathbb{Z}_M^*}$. Its image satisfies the conjugation constraint $z_{-j} = \overline{z_j}$, and we let $\tau_N : \sigma(\mathcal{R}) \to \mathbb{C}^{N/2}$ denote the projection that keeps one representative from each conjugate pair. The secret key is sampled from a sparse ternary distribution χ_{key} over $\mathcal{R}$, and errors are sampled from a discrete Gaussian χ_{err}. The encoding and encryption procedures are as follows.

- **Encode**. For a message $\mathbf{m} \in \mathbb{C}^{N/2}$, encoding returns the plaintext $\mathsf{pt} = \mathsf{Encode}(\mathbf{m}) = \lfloor \sigma^{-1}(\Delta \cdot \tau_N^{-1}(\mathbf{m})) \rceil \in \mathcal{R}$.
- **Decode**. For a plaintext $\mathsf{pt} \in \mathcal{R}$, decoding returns $\mathsf{Decode}(\mathsf{pt}) = \tau_N(\sigma(\mathsf{pt})/\Delta) \in \mathbb{C}^{N/2}$.
- **Encrypt**. For a plaintext $\mathsf{pt} \in \mathcal{R}$ and a secret key $\mathbf{s} \leftarrow \chi_{\mathsf{key}}$, encryption samples $a \leftarrow \mathcal{R}_{Q'_L}$ uniformly and an error $e \leftarrow \chi_{\mathsf{err}}$, and outputs $\mathsf{Enc}_{\mathbf{s}}(\mathsf{pt}) = \mathsf{ct} = (c_0, c_1) = (\mathsf{pt} - a\mathbf{s} + e,\ a) \bmod Q'_L$.
- **Decrypt**. Given $\mathsf{ct} = (c_0, c_1) \in \mathcal{R}^2_{Q'_\ell}$, decryption computes $\mathsf{pt}' = c_0 + c_1\mathbf{s} \bmod Q'_\ell$ and returns $\mathsf{Decode}(\mathsf{pt}')$, which recovers an approximation of $\mathbf{m}$.

CKKS supports homomorphic addition, scalar multiplication, ciphertext multiplication, and rescale, where a ciphertext multiplication is followed by a key switching and a rescaling that consumes one level. We use these operations as black boxes in the bootstrapping and defer their RNS-level operation to Appendix A.

Table 2. Notation used throughout the paper.

Symbol	Meaning
Sets and parameters	
$\mathbb{Z}_P$	Integers modulo the table size P
$\mathbb{R},\ \mathbb{C}$	Real and complex numbers
$\mathcal{R}_Q$	Ciphertext ring at modulus Q
$P = \alpha\beta$	Table size, rows α, columns β
$p,\ t$	Digit base, digit count $t = \lceil \log_p P \rceil$
Δ	CKKS scaling factor
ε	Target precision, $\varepsilon = 1/(2P)$
LUT matrix and its decomposition	
f	Function of the LUT, $f : \mathbb{Z}_P \to \mathbb{R}$
$\mathbf{M}_f$	LUT matrix, $\mathbf{M}_f[i,j] = f(i\beta + j)$
q	$\min\{\alpha, \beta\}$, maximal rank of $\mathbf{M}_f$
$\sigma_k,\ \mathbf{u}_k,\ \mathbf{v}_k$	Singular value and vectors of $\mathbf{M}_f$
$\mathbf{M}_f^{(r)}$	Rank-r truncated SVD of $\mathbf{M}_f$
$\widetilde{f}_r$	Rank-r LUT evaluation, $\mathbf{M}_f^{(r)}[i,j]$
r	Retained rank
$\boldsymbol{\delta}_{\mathrm{MSD}},\ \boldsymbol{\delta}_{\mathrm{LSD}}$	Row and column selectors
CKKS ciphertexts and operations	
ct	CKKS ciphertext
$\varepsilon(\mathsf{ct})$	Decryption noise of ct
MulScal	Scalar multiplication
Mul	Ciphertext multiplication
FDD	Functional digit decomposition
KProduct	Kronecker product of digit selectors
SlotsToCoeffs, ModSwitch	Slot-to-coefficient, modulus switch
$B_{\mathsf{in}},\ B_{\mathsf{rs}},\ B_{\mathsf{err}}$	Input, rescaling, multiplication noise

2.3 CKKS Functional Bootstrapping

CKKS bootstrapping [16] restores multiplicative depth by refreshing ciphertexts whose modulus has been consumed down to q_0'. Alexandru et al. [2] and Bae et al. [3] extend this procedure to evaluate an arbitrary function during bootstrapping. The input is an RLWE ciphertext encrypting an integer $x \in \mathbb{Z}_P$, and the output is a CKKS ciphertext encrypting $f(x)$ for a function $f : \mathbb{Z}_P \to \mathbb{R}$ specified by its lookup table $\mathsf{LUT}(f)$. Note that the input domain is the discrete set $\mathbb{Z}_P$, and a real input is first quantized to this grid, whereas the output $f(x)$ may be an arbitrary real number. The procedure relies on the first-order trigonometric

Hermite interpolant of f, denoted by T_f, which satisfies

$$\mathrm{Re}(T_f(\exp(2\pi \mathrm{i} k/P))) = f(k),$$
$$\mathrm{Re}\big(T_f'(\exp(2\pi \mathrm{i} k/P))\big) = 0, \quad \forall k \in \{0, \ldots, P-1\}.$$

Evaluating T_f yields f on the P-th roots of unity, and the vanishing first derivative achieves noise cleaning. The AKP-FBT [2] proceeds in six steps.

- **ModSwitch**. Convert the input RLWE ciphertext to a CKKS ciphertext by switching the modulus from q to q_0' and adjusting the scaling factor by Δ/q_0', which yields a CKKS ciphertext encoding $\Delta \cdot x/P$.
- **ModRaise** and **CoeffsToSlots**. Raise the ciphertext modulus to Q_L', yielding $t(X) = \Delta \cdot x/P + q_0' \cdot I(X)$. CoeffsToSlots then homomorphically applies the encoding map to convert each coefficient of $t(X)$ in a separate slot.
- **EvalExp**. Approximate $z = \exp(2\pi \mathrm{i}\, t_j/P)$ on each slot via Chebyshev interpolation [16], which removes the q_0'-overflow and reduces t_j modulo P.
- **EvalLUT**. Evaluate the trigonometric Hermite interpolant $T_f(z) = \alpha_0 + \sum_{t=1}^{P-1} \alpha_t\, z^t$ on the slot-wise outputs of EvalExp, which yields $f(x)$ in each slot.
- **SlotsToCoeffs**. Convert the slot-encoded result back to the coefficient encoding by homomorphically applying the decoding map with scaling factor $Q'/\Delta P$, which produces an encryption of $Q'/P \cdot \widetilde{m}(X)$, where Q' denotes the modulus after bootstrapping and $\widetilde{m}(X)$ encodes $f(x)$.
- **ModSwitch**. Switch the modulus from Q' to Q, which yields an RLWE ciphertext encrypting $Q/P \cdot \widetilde{m}(X)$ with ciphertext modulus Q.

The dominant depth cost arises from EvalLUT. Evaluating the degree-$(P-1)$ Hermite interpolant has multiplicative depth $\lceil \log_2(P-1) \rceil$, and the Paterson–Stockmeyer algorithm [35] reduces the number of ciphertext multiplications to $O(\sqrt{P})$.

2.4 CMT-FBT Building Blocks

CMT-FBT [25] builds on three extensions of the AKP-FBT described above, i.e., multi-value bootstrapping (MVB), functional digit decomposition (FDD), and Kronecker product (KProd).

Multi-value bootstrapping (MVB). MVB [11] evaluates k functions $f_1, \ldots, f_k$ on the same encrypted input within a single bootstrapping invocation, amortizing the computational cost of some steps. We write it as

$$\{\mathsf{ct}_i\}_{1 \le i \le k} \leftarrow \mathsf{MVB}(\mathsf{ct}, \{\mathsf{LUT}(f_i)\}_{1 \le i \le k}, p),$$

which takes an RLWE ciphertext ct encrypting $x \in \mathbb{Z}_p$ and the lookup tables of k functions $f_i : \mathbb{Z}_p \to \mathbb{R}$, and returns k ciphertexts encrypting $f_1(x), \ldots, f_k(x)$. MVB reuses some steps of AKP-FBT, sharing every step up to EvalExp across the k LUTs and branching only at the LUT-dependent steps.

– **Shared steps.** The steps such as ModSwitch, ModRaise, CoeffsToSlots, and EvalExp are executed once, producing $\exp(2\pi i x/p)$ in each slot.
– **EvalLUT (per LUT).** For each f_i, the trigonometric Hermite interpolant of f_i is evaluated from the shared powers of $\exp(2\pi i x/p)$, producing $f_i(x)$ in each slot.
– **SlotsToCoeffs and ModSwitch (per LUT).** Each result is mapped back to the coefficient encoding and switched to the output modulus Q, yielding k RLWE ciphertexts.

Algorithm 1: Functional digit decomposition (FDD).

procedure $\mathsf{FDD}(\mathsf{ct} \in \mathcal{R}_Q^2, \{\mathsf{LUT}(f_i)\}_{1\le i\le k}, p, P)$
  $t \leftarrow \lceil \log_p P \rceil$
  for $i \in \{1, \dots, t\}$ **do**
    $\{\mathsf{ctResult}_j\}_{0\le j\le k} \leftarrow \mathsf{SlotsMVB}(\mathsf{ct},\ \{\mathsf{LUT}(\mathrm{mod}_p)\} \cup \{\mathsf{LUT}(f_j)\}_j)$
    // output 0 is mod_p, outputs $1, \dots, k$ are $f_1, \dots, f_k$
    $\mathsf{ct}_i \leftarrow \{\mathsf{ctResult}_j\}_{0\le j\le k}$
    if $i < t$ **then**
      $\mathsf{ctMod} \leftarrow \mathsf{ModSwitch}(\mathsf{SlotsToCoeffs}(\mathsf{ctResult}_0), Q)$ // back to coefficient encoding
      $\mathsf{ct} \leftarrow \mathsf{ct} - \mathsf{ctMod}$
      $\mathsf{ct} \leftarrow \mathsf{ModSwitch}(\mathsf{ct}, Q/p)$
      $P \leftarrow P/\min(p, P/p)$
      $Q \leftarrow Q/\min(p, P/p)$
  return $\{\mathsf{ct}_i\}_{1\le i\le t}$

Functional digit decomposition (FDD). FDD [25] homomorphically decomposes an input in $\mathbb{Z}_P$ into radix-p digits and evaluates a set of functions on each extracted digit through MVB. We write it as

$$\{\mathsf{ct}_j\}_{1\le j\le t} \ \leftarrow\ \mathsf{FDD}(\mathsf{ct},\ \{\mathsf{LUT}(f_i)\}_{1\le i\le k},\ p,\ P),$$

which takes an RLWE ciphertext ct encrypting $x \in \mathbb{Z}_P$, the lookup tables of k functions $f_i : \mathbb{Z}_p \to \mathbb{R}$, a digit base p, and the table size P. Writing $x = \sum_{j=0}^{t-1} p^j x_j$ with $x_j \in \mathbb{Z}_p$ and $t = \lceil \log_p P \rceil$, it returns for each digit x_j a ciphertext encrypting the values $f_1(x_j), \dots, f_k(x_j)$ in slot encoding. The procedure extracts the digits from least to most significant digits, and the j-th iteration runs three steps. Note that CMT-FBT uses the slot-encoded variant SlotsMVB, which omits the SlotsToCoeffs step and keeps the k outputs in the slot encoding.
– **SlotsMVB.** A single SlotsMVB call evaluates, on the current digit x_j, the residue map mod_p together with the k functions $f_1, \dots, f_k$. The mod_p output recovers the digit x_j for the subsequent digit-removal step, and the remaining k outputs are the per-digit values returned by FDD. Each map is a LUT on $\mathbb{Z}_p$, so its interpolation degree depends on p rather than P.

– **Digit removal.** The mod_p output is returned to the coefficient encoding by SlotsToCoeffs and ModSwitch and subtracted from the running ciphertext, removing the extracted digit x_j. Then, a further ModSwitch divides the remainder by p, so that x_{j+1} becomes the least significant digit in the next iteration.

The last iteration omits steps 2 since no digit remains to extract. Algorithm 1 details the procedure.

To obtain the Kronecker-delta vector of each digit, the multiplexer tree bootstrapping [25] instantiates the f_i with the p coordinate maps $z \mapsto \delta_{z,a}$ for each $a \in \{0, \ldots, p-1\}$, where $\delta_{z,a}$ equals 1 when $z = a$ and 0 otherwise. The $k = p$ outputs then form

$$\mathbf{e}_{x_j} = (\delta_{x_j,0}, \ldots, \delta_{x_j,p-1})^\top \in \{0,1\}^p,$$

the x_j-th standard basis vector of $\mathbb{R}^p$.

Kronecker product (KProduct). KProduct procedure [25] combines the per-digit Kronecker-delta vectors of a group of digits into a single selector. We write it as

$$\boldsymbol{\delta} \leftarrow \mathsf{KProduct}(\{\mathsf{ct}_{e,\ell}\}_\ell),$$

which takes the ciphertexts of the digit vectors $\mathbf{e}_{x_\ell}$ and returns the ciphertext of their Kronecker product $\boldsymbol{\delta} = \bigotimes_\ell \mathbf{e}_{x_\ell}$. As each $\mathbf{e}_{x_\ell}$ is the x_ℓ-th standard basis vector, this product is the standard basis vector indexed by $\sum_\ell p^\ell x_\ell$. Applying KProduct separately to the high-order and low-order digit groups yields the row and column selectors $\boldsymbol{\delta}_{\mathrm{MSD}} \in \{0,1\}^\alpha$ and $\boldsymbol{\delta}_{\mathrm{LSD}} \in \{0,1\}^\beta$ that pick out the row and column of $\mathbf{M}_f$ indexed by the encrypted input. KProduct evaluates the product as a balanced binary tree of ciphertext multiplications, which keeps the multiplicative depth logarithmic in the number of digits. We provide detailed steps in Algorithm 6 of Appendix C.

3 Low Rank Structure of LUT Matrices

The dominant computational cost of CMT-FBT at high precision is the dense LUT matrix evaluation, i.e., the bilinear form $\boldsymbol{\delta}_{\mathrm{MSD}}^\top \mathbf{M}_f \boldsymbol{\delta}_{\mathrm{LSD}}$, where $\boldsymbol{\delta}_{\mathrm{MSD}} \in \{0,1\}^\alpha$ and $\boldsymbol{\delta}_{\mathrm{LSD}} \in \{0,1\}^\beta$ are the row and column selectors of the $\alpha \times \beta$ LUT matrix $\mathbf{M}_f$. In this section, we show that the LUT matrices induced by many practical functions are low-rank. Then, we introduce two mechanisms for the low-rank structure, algebraic separability and analytic smoothness, each reflected in the singular-value decay of $\mathbf{M}_f$. Accordingly, LUTs fall into three spectral classes, exactly low-rank (Class A), numerically low-rank (Class B), and spectrally flat (Class C).

3.1 Singular Value Decomposition of LUT Matrices

For a LUT $f : \mathbb{Z}_P \to \mathbb{R}$, we factor $P = \alpha\beta$ and take the balanced choice $\alpha = \beta = \sqrt{P}$ by default, though the analysis holds for any factorization of P.

Following the matrix representation used in CMT-FBT [25], each input $x \in \mathbb{Z}_P$ is written uniquely as $x = i\beta + j$ with $i = \lfloor x/\beta \rfloor$ and $j = x \bmod \beta$. This decomposition reshapes the LUT vector $\mathbf{y}_f = (f(0), \ldots, f(P-1))^\top$ into an $\alpha \times \beta$ matrix $\mathbf{M}_f$ defined by

$$\mathbf{M}_f[i, j] = f(i\beta + j).$$

The unfolding $x = i\beta + j$ preserves locality within each row of $\mathbf{M}_f$, mapping nearby inputs to neighboring entries except across row boundaries. Consequently, a smooth f induces a matrix with entries that vary gradually across rows and columns, thereby admitting accurate low-rank approximation. Swapping the two digit groups between the row and column indices only transposes $\mathbf{M}_f$ and preserves its rank. In contrast, an arbitrary reindexing of $\mathbb{Z}_P$ destroys this regularity. We analyze the rank structure of $\mathbf{M}_f$ via its singular value decomposition. Since f is public, this decomposition is computed once in plaintext, and only the rank-r factors are used in homomorphic evaluation.

Definition 1 (Singular value decomposition of the LUT matrix). *Let $\mathbf{M}_f \in \mathbb{R}^{\alpha \times \beta}$ be the LUT matrix induced by f, and let $q = \min\{\alpha, \beta\}$. By the standard existence theorem for real matrices [28], $\mathbf{M}_f$ has a singular value decomposition*

$$\mathbf{M}_f = \sum_{k=1}^{q} \sigma_k \mathbf{u}_k \mathbf{v}_k^\top,$$

where $\sigma_1 \geq \sigma_2 \geq \cdots \geq \sigma_q \geq 0$ are the singular values of $\mathbf{M}_f$, and $\{\mathbf{u}_k\} \subset \mathbb{R}^\alpha$, $\{\mathbf{v}_k\} \subset \mathbb{R}^\beta$ are orthonormal families of left and right singular vectors.

If $\operatorname{rank}(\mathbf{M}_f) = r_0 \ll q$, the first r_0 singular components reproduce $\mathbf{M}_f$ exactly. More generally, when the singular values decay rapidly, a rank-r truncation with $r \ll q$ suffices to approximate $\mathbf{M}_f$ accurately. In either case, the approximation achieves precision ε provided that $\sigma_{r+1}(\mathbf{M}_f) \leq \varepsilon$.

Definition 2 (Truncated singular value decomposition of the LUT matrix). *For a rank parameter $r \leq q$, the rank-r truncated SVD of the LUT matrix retains the r leading terms,*

$$\mathbf{M}_f^{(r)} = \sum_{k=1}^{r} \sigma_k \mathbf{u}_k \mathbf{v}_k^\top.$$

By the Eckart–Young theorem [28], $\mathbf{M}_f^{(r)}$ is the rank-r approximation of $\mathbf{M}_f$ in the spectral norm,

$$\|\mathbf{M}_f - \mathbf{M}_f^{(r)}\|_2 = \min_{\operatorname{rank}(A) \leq r} \|\mathbf{M}_f - A\|_2.$$

For an input $x = i\beta + j$, the corresponding rank-r LUT evaluation is

$$\widetilde{f}_r(x) = \mathbf{M}_f^{(r)}[i, j] = \sum_{k=1}^{r} \sigma_k \mathbf{u}_k[i] \mathbf{v}_k[j].$$

If $\mathbf{M}_f$ has rank at most r, then no nonzero singular value is discarded, and the evaluation is exact, i.e., $\widetilde{f}_r(x) = f(x)$ for all $x \in \mathbb{Z}_P$. Otherwise, $\widetilde{f}_r(x)$ approximates $f(x)$, with an error controlled by the discarded singular values.

Theorem 1 (Error of truncated SVD). *Consider the singular value decomposition $\mathbf{M}_f = \sum_{k=1}^{q} \sigma_k \mathbf{u}_k \mathbf{v}_k^\top$ of the LUT matrix $\mathbf{M}_f$, where $q = \min\{\alpha, \beta\}$ and $\sigma_1 \geq \cdots \geq \sigma_q \geq 0$. For a given rank parameter $r \leq q$, the rank-r truncation $\mathbf{M}_f^{(r)}$ induces the approximate evaluation $\widetilde{f}_r(x) = \mathbf{M}_f^{(r)}[i, j]$ for $x = i\beta + j$. Then, for every $x \in \mathbb{Z}_P$,*

$$|f(x) - \widetilde{f}_r(x)| = |(\mathbf{M}_f - \mathbf{M}_f^{(r)})[i, j]| \leq \sigma_{r+1}(\mathbf{M}_f),$$

with the convention $\sigma_{q+1}(\mathbf{M}_f) = 0$ when $r = q$. In particular, if $\operatorname{rank}(\mathbf{M}_f) \leq r$, then $\mathbf{M}_f^{(r)} = \mathbf{M}_f$, and the rank-$r$ LUT evaluation is exact for every input.

Proof. Write $E = \mathbf{M}_f - \mathbf{M}_f^{(r)}$ for the truncation residual. For $x = i\beta + j$,

$$f(x) - \widetilde{f}_r(x) = E[i, j].$$

Expressing the (i, j)-entry through the standard basis vectors $\mathbf{e}_i \in \mathbb{R}^\alpha$ and $\mathbf{e}_j \in \mathbb{R}^\beta$ gives $E[i, j] = \mathbf{e}_i^\top E \mathbf{e}_j$. Because $\mathbf{e}_i$ and $\mathbf{e}_j$ are unit vectors, the Cauchy–Schwarz inequality and the definition of the spectral norm yield

$$|E[i, j]| = |\mathbf{e}_i^\top E \mathbf{e}_j| \leq \|\mathbf{e}_i\|_2 \, \|E\mathbf{e}_j\|_2 \leq \|\mathbf{e}_i\|_2 \, \|E\|_2 \, \|\mathbf{e}_j\|_2 = \|E\|_2,$$

where $\|E\|_2$ denotes the spectral norm of E. It remains to identify $\|E\|_2$. By construction,

$$E = \sum_{k=r+1}^{q} \sigma_k \mathbf{u}_k \mathbf{v}_k^\top.$$

The sets $\{\mathbf{u}_k\}_{k>r}$ and $\{\mathbf{v}_k\}_{k>r}$ remain orthonormal, so this expression is itself an SVD of E with largest singular value $\sigma_{r+1}(\mathbf{M}_f)$. Since the spectral norm of a matrix equals its largest singular value, $\|E\|_2 = \sigma_{r+1}(\mathbf{M}_f)$, with the convention $\sigma_{q+1}(\mathbf{M}_f) = 0$ when $r = q$, and therefore

$$|f(x) - \widetilde{f}_r(x)| \leq \sigma_{r+1}(\mathbf{M}_f).$$

When $\operatorname{rank}(\mathbf{M}_f) \leq r$, all singular values beyond the r-th vanish, giving $E = 0$ and hence $\mathbf{M}_f^{(r)} = \mathbf{M}_f$, so the rank-$r$ evaluation is exact for every input.

3.2 Exact Low Rank Structure in LUT Matrices

The truncated SVD is exact when $\mathbf{M}_f$ has rank at most r. This holds whenever f is separable on the index grid, that is, when $f(i\beta + j)$ factors as a finite sum of products of a row term and a column term. We define such functions as *exactly low-rank* (Class A).

Lemma 1 (Separable LUTs). *Let $\mathbf{M}_f$ be the $\alpha \times \beta$ LUT matrix of f. If*

$$\mathbf{M}_f[i,j] = \sum_{\ell=1}^{r} g_\ell(i)\, h_\ell(j)$$

for all i, j, then $\operatorname{rank}(\mathbf{M}_f) \le r$.

Proof. Let $\mathbf{g}_\ell = (g_\ell(0), \ldots, g_\ell(\alpha-1))^\top$ and $\mathbf{h}_\ell = (h_\ell(0), \ldots, h_\ell(\beta-1))^\top$. The hypothesis can be written as $\mathbf{M}_f = \sum_{\ell=1}^{r} \mathbf{g}_\ell \mathbf{h}_\ell^\top$, which is a sum of r rank-one matrices. Hence, $\operatorname{rank}(\mathbf{M}_f) \le r$.

Lemma 1 formalizes the classical fact that a sum of r rank-one terms has rank at most r in matrix form. While this result is general, the separability of practical LUTs typically arises from one of two structural forms. The first is a partition of the domain into affine pieces, where the rank is determined by the number of pieces rather than P. We address this case first, before turning to separability derived from algebraic identities.

Corollary 1 (Piecewise-affine LUTs). *Let $f : \mathbb{Z}_P \to \mathbb{R}$ be piecewise-affine with K pieces, and let $\mathbf{M}_f$ be its LUT matrix under the decomposition $x = i\beta + j$. Let m denote the number of rows of $\mathbf{M}_f$ that contain a breakpoint of f. Then*

$$\operatorname{rank}(\mathbf{M}_f) \le \begin{cases} 1+m & \text{if } f \text{ is piecewise-constant,} \\ 2+m & \text{in general.} \end{cases}$$

In both cases, $m \le K-1$, yielding the bound $\operatorname{rank}(\mathbf{M}_f) \le K+1$ that depends only on the number of pieces instead of the table size P.

Proof. We split the rows of $\mathbf{M}_f$ into those interior to a single affine piece of f and those containing a breakpoint, and bound the rank contributed by each part. For the interior rows, suppose row i lies entirely in the k-th piece, where $f(x) = a_k x + b_k$. Its entries have

$$\mathbf{M}_f[i,j] = a_k(i\beta + j) + b_k = c_i + a_k\, j,$$

for $c_i = a_k \beta i + b_k$. Each such row equals $c_i \mathbf{1} + a_k \boldsymbol{\nu}$, where $\mathbf{1} = (1, \ldots, 1)^\top$ and $\boldsymbol{\nu} = (0, 1, \ldots, \beta-1)^\top$, are fixed and independent of which piece i belongs to. Therefore, all interior rows lie in $\operatorname{span}\{\mathbf{1}, \boldsymbol{\nu}\}$ and contribute rank at most 2 in total. Each of the m breakpoint rows contributes rank at most 1, so $\operatorname{rank}(\mathbf{M}_f) \le 2+m$. Since every breakpoint of f falls in exactly one row of $\mathbf{M}_f$, we also have $m \le K-1$. When f is piecewise-constant, every a_k vanishes, and the term $a_k \boldsymbol{\nu}$ disappears, so these rows span only $\mathbf{1}$. The rank contribution from this group drops from 2 to 1, tightening the bound to $\operatorname{rank}(\mathbf{M}_f) \le 1+m$.

The bound is tight on some common functions, where the only remaining freedom is how the breakpoints of f align with the row boundaries of $\mathbf{M}_f$.

Example 1 (Common piecewise-affine functions). The bound depends on whether each breakpoint of f falls on a row boundary or inside a row of $\mathbf{M}_f$. A breakpoint on a row boundary leaves every row interior to a single piece, while one inside a row turns that row into a breakpoint row and contributes one extra unit to the bound.

- *Step.* The step function is defined as $f(x) = 1$ for $x \geq t$ and $f(x) = 0$ otherwise. It has two constant pieces, so $\operatorname{rank}(\mathbf{M}_f) = 1$ when t lies on a row boundary and $\operatorname{rank}(\mathbf{M}_f) = 2$ otherwise.
- *Sign.* $\operatorname{sign}(x-t) = 2 \cdot \operatorname{Step}(x-t) - 1$ has the same rank as the step function.
- *ReLU.* $f(x) = \max\{0, x - P/2\}$ has two affine pieces, so $\operatorname{rank}(\mathbf{M}_f) = 2$ when the breakpoint lies on a row boundary and $\operatorname{rank}(\mathbf{M}_f) \leq 3$ otherwise.
- *Absolute value.* $f(x) = |x - P/2|$ has two affine pieces and the same rank as ReLU.

Separability can also come from an algebraic identity that splits $f(i\beta + j)$ into row and column terms directly, without any partition of the domain.

Example 2 (Separability from algebraic identities). Each function below splits through such an identity, with a rank independent of P.

- *Exponential.* $\exp(i\beta + j) = \exp(i\beta)\exp(j)$ gives the rank-one factorization $\mathbf{M}_f = \mathbf{g}\mathbf{h}^\top$ with $g(i) = \exp(i\beta)$ and $h(j) = \exp(j)$, so $\operatorname{rank}(\mathbf{M}_f) = 1$.
- *Sine and cosine.* The angle-addition identity $\sin(i\beta + j) = \sin(i\beta)\cos(j) + \cos(i\beta)\sin(j)$ produces two rank-one terms, so $\operatorname{rank}(\mathbf{M}_f) \leq 2$, and likewise for cos.
- *Monomial.* For $f(x) = x^m$ the binomial expansion $(i\beta+j)^m = \sum_{a=0}^{m} \binom{m}{a} (i\beta)^{m-a} j^a$ splits into $m + 1$ terms, so $\operatorname{rank}(\mathbf{M}_f) \leq m + 1$.
- *Modular reduction.* For $f(x) = x \bmod k$ with $k \mid \beta$ the value is $j \bmod k$ and depends on the column alone, so $\operatorname{rank}(\mathbf{M}_f) = 1$. The case $k \nmid \beta$ admits a similar low-rank decomposition, with the rank bounded by a small constant determined by $\gcd(k, \beta)$.
- *Floor division.* For $f(x) = \lfloor x/\beta \rfloor$ the value is i and depends on the row alone, so $\operatorname{rank}(\mathbf{M}_f) = 1$.

3.3 Numerically Low Rank LUTs

A LUT matrix that is not exactly low-rank can still be *numerically* low-rank, with singular values that decay geometrically rather than vanishing after finitely many terms. This holds whenever f extends analytically to a complex neighborhood of the sampling interval, which covers the smooth activations Sigmoid, tanh, GeLU, and SiLU. We call such functions *numerically low rank* (Class B). By Theorem 1, a rank-r truncation reaches a target precision ε once $\sigma_{r+1}(\mathbf{M}_f) \leq \varepsilon$. The key question is then how to determine the minimal r that satisfies this condition.

The threshold ε is determined by the table size P, where the table of size P resolves its input to $\log_2 P$ bits. To attain the same accuracy on the output,

we normalize the output range of f to $[0,1]$, in which one least-significant bit equals $1/P$. Correct rounding then requires the output to be recovered within half a least-significant bit, so the entrywise approximation error of $\mathbf{M}_f$ must stay below $1/(2P)$, which we take as the CKKS correctness threshold $\varepsilon = 1/(2P)$.

Theorem 2 (Exponential singular-value decay). *Let $g : [0,1] \to \mathbb{R}$ be analytic in an open complex neighborhood of $[0,1]$, and let $f : \mathbb{Z}_P \to \mathbb{R}$ be its sampled version $f(x) = g(x/P)$. Let $P = \alpha\beta$, so that $\mathbf{M}_f \in \mathbb{R}^{\alpha\times\beta}$ has entries $\mathbf{M}_f[i,j] = g\left(\frac{i\beta+j}{P}\right)$ for $0 \le i < \alpha$ and $0 \le j < \beta$. Then there exist constants $\rho > 1$ and $M > 0$, depending only on g and its region of analyticity, such that*

$$\sigma_{r+1}(\mathbf{M}_f) \le \frac{2M\sqrt{\alpha\beta}}{\rho - 1}\,\rho^{-(r-1)}, \qquad r \ge 1.$$

The proof constructs a rank-r approximation from the Chebyshev truncation of g on a Bernstein ellipse [36] and bounds $\sigma_{r+1}(\mathbf{M}_f)$ through the Eckart–Young theorem. The dimension factor $\sqrt{\alpha\beta}$ admits a tighter, dimension-free form for matrices with displacement structure, which we defer to Appendix D together with the full proof.

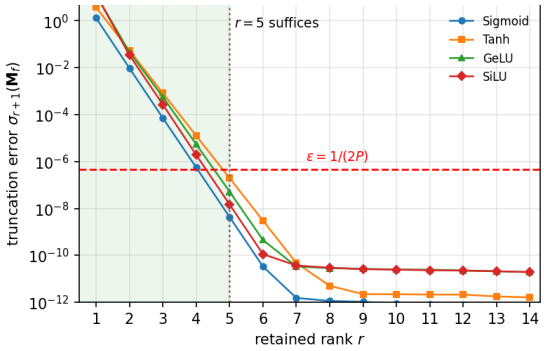


Fig. 2. Truncation error $\sigma_{r+1}(\mathbf{M}_f)$ of the exact LUT matrix as a function of the retained rank r, for Sigmoid, tanh, GeLU, and SiLU at $P = 2^{20}$ over the input range $[-64, 64]$. The error decays exponentially, consistent with Theorem 2. A rank of $r = 5$ already drives the error below the CKKS correctness threshold $\varepsilon = 1/(2P)$ for all four activations.

Solving $\sigma_{r+1}(\mathbf{M}_f) \le \varepsilon$ for the conservative bound of Theorem 2 at $\varepsilon = 1/(2P)$ gives $r = \frac{3}{2\ln\rho}\ln P + O(1)$, where the factor $\frac{3}{2}$ collects the $\sqrt{P}$ term of the bound and the $1/(2P)$ threshold. The decay rate ρ depends only on the input range over which f is sampled, and is independent of the table size P. Because ρ does not depend on P, the required rank grows only logarithmically in P, rather than linearly with the $\sqrt{P}$ rows of $\mathbf{M}_f$, so the evaluation cost reduces from $O(P)$ to $O(\sqrt{P}\log P)$. A wider input range moves the singularities of g toward the real

axis, which lowers ρ and raises r by a small amount. For common activations, the small constant $1/\ln\rho$ ensures that the required rank remains substantially below its asymptotic estimate. As illustrated in Figure 2, for $P = 2^{20}$, the truncation error of all four activations falls below $\varepsilon = 1/(2P)$ at $r = 5$, which is two orders of magnitude smaller than the matrix dimension $\sqrt{P} = 1024$.

Remark 1 (Effect of finite-precision encoding). The decay analysis above applies to the exact matrix $\mathbf{M}_f$, whereas CKKS encodes each entry at a finite scale Δ and may perturb the numerical rank. Rounding each entry to a multiple of $1/\Delta$ yields an error matrix E with $\|E\|_{\max} \le 1/(2\Delta)$, and hence $\|E\|_2 \le \sqrt{P}/(2\Delta)$. By Weyl's inequality [32, Theorem 4.3.1], each singular value shifts by at most $\|E\|_2$, so $\sigma_{r+1}(\mathbf{M}_f + E) \le \sigma_{r+1}(\mathbf{M}_f) + \|E\|_2$. The numerical rank at threshold ε is preserved once $\sqrt{P}/(2\Delta) \le \varepsilon$. At $\varepsilon = 1/(2P)$ this reduces to $\log_2 \Delta \gtrsim \frac{3}{2}\log_2 P$, which requires $\log_2 \Delta \gtrsim 30$ at the largest precision $P = 2^{20}$ and is met by the scaling factors $\Delta \ge 2^{40}$ used in practice. The encoded spectrum then matches the exact one up to negligible error and preserves the exact rank. A smaller Δ instead lifts the trailing singular values to the order of $\sqrt{P}/(2\Delta)$, and once this floor exceeds ε the numerical rank inflates, as illustrated in Figure 3.

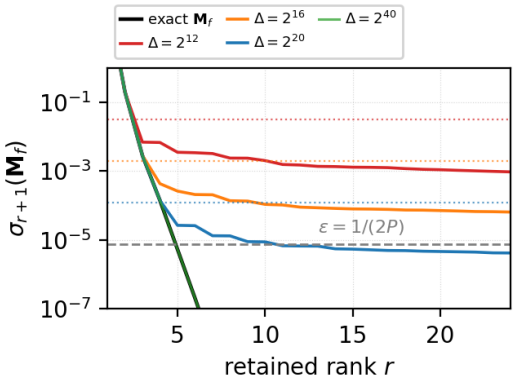


Fig. 3. Singular-value spectrum of the GeLU ($P = 2^{16}$, representative example) and its finite-precision encodings at scaling factors $\Delta \in \{2^{12}, 2^{16}, 2^{20}, 2^{40}\}$. When this floor lies above the threshold $\varepsilon = 1/(2P)$ (dashed line), the numerical rank is inflated. With $\Delta \ge 2^{40}$ under practical parameter settings, the noise floor remains far below ε (outside the plotted range), so the encoded spectrum aligns with the exact one (black) and crosses ε at the same rank, preserving the exact numerical rank.

3.4 Spectrally Flat LUTs

Some LUT matrices exhibit no exploitable low-rank structure. Their singular values remain elevated across the entire spectrum, preventing any rank-r trun-

cation with $r \ll \min\{\alpha, \beta\}$ from achieving the target precision ε. We term such functions *spectrally flat* (Class C).

Example 3 (AES S-box). The AES S-box over $\mathbb{F}_{2^8}$, when arranged as a 16×16 matrix, is full rank with singular values exhibiting no significant decay. Consequently, any rank-r truncation with $r < 16$ incurs an error on the order of $\sigma_{\min}(\mathbf{M}_f)$, which remains well above practical precision thresholds ε.

Example 4 (Uniform random LUT). Let $f : \mathbb{Z}_P \to \mathbb{R}$ assign an independent uniform random value to each input, and reshape its table into the $\alpha \times \beta$ matrix. Similar to AES S-Box, $\mathbf{M}_f$ is also full-rank, exhibiting no decay.

Summary. The singular-value spectrum of $\mathbf{M}_f$ partitions LUTs into three classes, each determining the attainable rank reduction. Class A (*exactly low-rank*) comprises functions whose spectra terminate after finitely many terms, including piecewise-affine maps, separable functions (e.g., exp and sin), and modular or bitwise operations. Class B (*numerically low-rank*) encompasses smooth functions, such as Sigmoid, tanh, GeLU, and SiLU, whose spectra exhibit exponential decay and thus admit truncation at small ranks. Class C (*spectrally flat*) consists of unstructured LUT such as AES S-boxes, whose spectra remain elevated throughout and preclude low-rank approximation. Together, these three classes characterize the spectral compressibility of $\mathbf{M}_f$ and determine whether exact, approximate, or no acceleration is feasible.

4 Low Rank Multiplexer Tree Functional Bootstrapping

Building on the spectral framework of Section 3, we propose the low rank multiplexer tree functional bootstrapping for structured LUTs. We first present the low rank LUT matrix evaluation, then integrate it into the full LRMT-FBT pipeline, and finally describe three extensions.

4.1 Low Rank LUT Matrix Evaluation

We present a low-rank matrix evaluation that reduces the cost of the LUT evaluation step in CMT-FBT by exploiting the spectral structure of $\mathbf{M}_f$. Recall from Section 3.1 that the LUT of $f : \mathbb{Z}_P \to \mathbb{R}$ is organized as a matrix $\mathbf{M}_f \in \mathbb{R}^{\alpha \times \beta}$ with entries $\mathbf{M}_f[i, j] = f(i\beta + j)$, where SVD is $\mathbf{M}_f = \sum_{k=1}^{q} \sigma_k \, \mathbf{u}_k \mathbf{v}_k^\top$ and $q = \min\{\alpha, \beta\}$. For an encrypted input $x = i\beta + j \in \mathbb{Z}_P$, the FDD and KProduct phases yield the encrypted row and column selectors $\boldsymbol{\delta}_{\mathrm{MSD}} \in \{0, 1\}^\alpha$ and $\boldsymbol{\delta}_{\mathrm{LSD}} \in \{0, 1\}^\beta$ as described in Section 2.4, expressing the LUT as the bilinear form

$$f(x) \;=\; \boldsymbol{\delta}_{\mathrm{MSD}}^\top \mathbf{M}_f \, \boldsymbol{\delta}_{\mathrm{LSD}}, \tag{1}$$

where CMT-FBT computes this form directly through $\alpha\beta = P$ scalar multiplications. The singular value decomposition of $\mathbf{M}_f$ decomposes the bilinear form in

Equation (1) into a sum of separable terms. Specifically, substituting the rank-r truncation $\mathbf{M}_f^{(r)} = \sum_{k=1}^{r} \sigma_k \, \mathbf{u}_k \mathbf{v}_k^\top$ yields

$$\widetilde{f}_r(x) \;:=\; \boldsymbol{\delta}_{\mathrm{MSD}}^\top \mathbf{M}_f^{(r)} \, \boldsymbol{\delta}_{\mathrm{LSD}} \;=\; \sum_{k=1}^{r} \sigma_k \big(\mathbf{u}_k^\top \boldsymbol{\delta}_{\mathrm{MSD}}\big)\big(\mathbf{v}_k^\top \boldsymbol{\delta}_{\mathrm{LSD}}\big).$$

We show the homomorphic evaluation procedure in Algorithm 2 based on this separable decomposition. Specifically, for each rank-k component, the algorithm first computes two encrypted inner products ct_{a_k} and ct_{b_k} via scalar multiplications, which encrypt and encode $\sqrt{\sigma_k}\,\mathbf{u}_k^\top \boldsymbol{\delta}_{\mathrm{MSD}}$ and $\sqrt{\sigma_k}\,\mathbf{v}_k^\top \boldsymbol{\delta}_{\mathrm{LSD}}$, respectively. Then, the single ciphertext multiplication yields ct_{c_k}, encrypting the k-th rank-one term. Summing these r terms produces the output ciphertext $\mathsf{ct}_{\mathrm{out}}$ that encrypts $\widetilde{f}_r(x)$.

Algorithm 2: LRMTEval: Low rank LUT matrix evaluation.

Input: Row and column selector ciphertexts $\{\mathsf{ct}_{\mathrm{MSD},i}\}_{i\in\{0,\ldots,\alpha-1\}}$ and $\{\mathsf{ct}_{\mathrm{LSD},j}\}_{j\in\{0,\ldots,\beta-1\}}$, encrypting the entries of the 0/1 selectors $\boldsymbol{\delta}_{\mathrm{MSD}} \in \{0,1\}^\alpha$ and $\boldsymbol{\delta}_{\mathrm{LSD}} \in \{0,1\}^\beta$ for an input $x = i\beta + j \in \mathbb{Z}_P$.
Precomputed rank-r factors $\{(\sigma_k, \mathbf{u}_k, \mathbf{v}_k)\}_{k=1}^r$ of $\mathbf{M}_f$.
Output: Ciphertext $\mathsf{ct}_{\mathrm{out}}$ encrypting $\widetilde{f}_r(x)$.

for $k = 1, \ldots, r$ **do**
  $\mathsf{ct}_{a_k} \leftarrow \sum_{i=0}^{\alpha-1} \mathsf{MulScal}\big(\mathsf{ct}_{\mathrm{MSD},i},\, \sqrt{\sigma_k}\,\mathbf{u}_k[i]\big)$ // encrypts $\sqrt{\sigma_k}\,\mathbf{u}_k^\top \boldsymbol{\delta}_{\mathrm{MSD}}$
  $\mathsf{ct}_{b_k} \leftarrow \sum_{j=0}^{\beta-1} \mathsf{MulScal}\big(\mathsf{ct}_{\mathrm{LSD},j},\, \sqrt{\sigma_k}\,\mathbf{v}_k[j]\big)$ // encrypts $\sqrt{\sigma_k}\,\mathbf{v}_k^\top \boldsymbol{\delta}_{\mathrm{LSD}}$
  $\mathsf{ct}_{c_k} \leftarrow \mathsf{Mul}(\mathsf{ct}_{a_k}, \mathsf{ct}_{b_k})$
$\mathsf{ct}_{\mathrm{out}} \leftarrow \sum_{k=1}^{r} \mathsf{ct}_{c_k}$
return $\mathsf{ct}_{\mathrm{out}}$

Computational complexity. Algorithm 2 requires $r(\alpha + \beta)$ scalar multiplications and r ciphertext multiplications, consuming two multiplicative levels. The additional level compared to CMT-FBT arises because the singular vector components $\sqrt{\sigma_k}\,\mathbf{u}_k[i]$ are real-valued. Thus, each scalar multiplication must be followed by a Rescale operation through one level, whereas the integer entries of $\mathbf{M}_f$ in CMT-FBT do not involve Rescale. In addition, the r rank-one terms share the same selector ciphertexts, so the depth is independent of r. For comparison, CMT-FBT [25] requires P scalar multiplications and $\sqrt{P}$ ciphertext multiplications in the balanced setting. Our LRMT-FBT reduces the number of scalar multiplications from P to $2r\sqrt{P}$ and the number of ciphertext multiplications from $\sqrt{P}$ to r as shown in Table 3.

Noise growth of LRMTEval. We analyze the noise growth of Algorithm 2. Let B_{in} bound the noise $\varepsilon_{\mathrm{in}}$ of each selector entry, and B_{rs} denote the noise from a single Rescale operation. B_{err} is the total relinearization and rescaling error per ciphertext multiplication. For each $k \in \{1, \ldots, r\}$, since $\boldsymbol{\delta}_{\mathrm{MSD}}$ and $\boldsymbol{\delta}_{\mathrm{LSD}}$ are

Table 3. Cost comparison of the LUT matrix evaluation stage. MulScal denotes scalar multiplications, Mul denotes ciphertext multiplications, and Levels denotes the multiplicative depth consumed.

Method	MulScal	Mul	Levels
CMT-FBT [25]	$\alpha\beta$	α	1
LRMT-FBT	$r(\alpha+\beta)$	r	2

binary selector vectors, we have

$$|\mathbf{u}_k^\top \boldsymbol{\delta}_{\mathrm{MSD}}| \le \|\mathbf{u}_k\|_\infty \le \|\mathbf{u}_k\|_2 = 1,$$

which implies $|a_k|, |b_k| \le \sqrt{\sigma_k}$. Since ct_{a_k} and ct_{b_k} result from α and β scalar multiplications, respectively, their noise is bounded by

$$|\varepsilon(\mathsf{ct}_{a_k})| \le \sqrt{\sigma_k}\,\|\mathbf{u}_k\|_1 B_{\mathsf{in}} + \alpha B_{\mathsf{rs}},$$
$$|\varepsilon(\mathsf{ct}_{b_k})| \le \sqrt{\sigma_k}\,\|\mathbf{v}_k\|_1 B_{\mathsf{in}} + \beta B_{\mathsf{rs}}.$$

The subsequent ciphertext multiplication $\mathsf{ct}_{c_k} = \mathsf{Mul}(\mathsf{ct}_{a_k}, \mathsf{ct}_{b_k})$ amplifies the noise according to

$$|\varepsilon(\mathsf{ct}_{c_k})| \le |a_k||\varepsilon(\mathsf{ct}_{b_k})| + |b_k||\varepsilon(\mathsf{ct}_{a_k})| + \tfrac{|\varepsilon(\mathsf{ct}_{a_k})\varepsilon(\mathsf{ct}_{b_k})|}{\Delta} + B_{\mathsf{err}}.$$

Under the parameter constraints $|\varepsilon(\mathsf{ct}_{a_k})\varepsilon(\mathsf{ct}_{b_k})|/\Delta \le B_{\mathsf{err}}$ and $B_{\mathsf{rs}} \le B_{\mathsf{err}}$, substituting the preceding bounds gives

$$|\varepsilon(\mathsf{ct}_{c_k})| \le \sigma_k\big(\|\mathbf{u}_k\|_1 + \|\mathbf{v}_k\|_1\big)B_{\mathsf{in}} + \big(\sqrt{\sigma_k}(\alpha+\beta) + 2\big)B_{\mathsf{err}}.$$

Summing over the r rank-one terms yields the output noise bound

$$|\varepsilon(\mathsf{ct}_{\mathrm{out}})| \le \sum_{k=1}^{r} \sigma_k\big(\|\mathbf{u}_k\|_1 + \|\mathbf{v}_k\|_1\big)B_{\mathsf{in}} + \Big((\alpha+\beta)\sum_{k=1}^{r}\sqrt{\sigma_k} + 2r\Big)B_{\mathsf{err}}.$$

4.2 The Full LRMT-FBT Algorithm

In this section, we introduce LRMT-FBT that builds on the multiplexer-tree construction of CMT-FBT [25]. It retains the FDD and KProduct stages that generate the multiplexer tree, and introduces a new low-rank LUT evaluation (Algorithm 2) in place of the dense matrix evaluation that dominates CMT-FBT at high precision (Figure 1). The full procedure is given in Algorithm 3. The FDD stage extracts the radix-p digits of x as encrypted Kronecker-delta vectors. Two KProduct calls then aggregate these vectors into the row and column selectors $\boldsymbol{\delta}_{\mathrm{MSD}}$ and $\boldsymbol{\delta}_{\mathrm{LSD}}$, split at index $s = \lceil t/2 \rceil$ among the $t = \lceil \log_p P \rceil$ digits. Our low-rank evaluation LRMTEval then takes these selectors together with the precomputed rank-r factors and returns $\widetilde{f}_r(x)$ in slot encoding, after which SlotsToCoeffs and a modulus switch produce an RLWE ciphertext.

Algorithm 3: The full LRMT-FBT pipeline.

Input: RLWE ciphertext $\mathsf{ct} \in \mathcal{R}_Q^2$ encrypting $x \in \mathbb{Z}_P$.
Digit base p, table size P.
Precomputed rank-r factors $\{(\sigma_k, \mathbf{u}_k, \mathbf{v}_k)\}_{k=1}^{r}$ of $\mathbf{M}_f$.
Output: RLWE ciphertext encrypting $\widetilde{f}_r(x)$ at modulus Q.

$t \leftarrow \lceil \log_p P \rceil$
$s \leftarrow \lceil t/2 \rceil$
$\{\mathsf{ct}_{e,\ell}\}_{\ell \in \{0,\dots,t-1\}} \leftarrow \mathsf{FDD}(\mathsf{ct}, \{\mathsf{LUT}(\delta_{\cdot,a})\}_{a \in \{0,\dots,p-1\}}, p, P)$
// Kronecker-delta vector $\mathbf{e}_{x_\ell}$ of digit ℓ, Algorithm 1
if *cleaning enabled* **then** // optional cleaning, used when P is large, Remark 2
  $\{\mathsf{ct}_{e,\ell}\}_{\ell \in \{0,\dots,t-1\}} \leftarrow \mathsf{Clean}\big(\{\mathsf{ct}_{e,\ell}\}_{\ell \in \{0,\dots,t-1\}}\big)$
$\{\mathsf{ct}_{\mathrm{LSD},j}\}_{j \in \{0,\dots,\beta-1\}} \leftarrow \mathsf{KProduct}\big(\{\mathsf{ct}_{e,\ell}\}_{\ell \in \{0,\dots,s-1\}}\big)$ // low-order Kronecker-delta vectors form the column selector, Algorithm 6
$\{\mathsf{ct}_{\mathrm{MSD},i}\}_{i \in \{0,\dots,\alpha-1\}} \leftarrow \mathsf{KProduct}\big(\{\mathsf{ct}_{e,\ell}\}_{s \le \ell < t}\big)$ // high-order Kronecker-delta vectors form the row selector,Algorithm 6
$\mathsf{ct}_{\mathrm{slot}} \leftarrow \mathsf{LRMTEval}\big(\{\mathsf{ct}_{\mathrm{MSD},i}\}, \{\mathsf{ct}_{\mathrm{LSD},j}\}, \{(\sigma_k, \mathbf{u}_k, \mathbf{v}_k)\}_{k=1}^{r}\big)$ // rank-r evaluation, Algorithm 2
$\mathsf{ct}_{\mathrm{coeff}} \leftarrow \mathsf{SlotsToCoeffs}(\mathsf{ct}_{\mathrm{slot}})$ // back to coefficient encoding
return $\mathsf{ModSwitch}(\mathsf{ct}_{\mathrm{coeff}}, Q)$ // RLWE encryption of $\widetilde{f}_r(x)$ at modulus Q

Remark 2 (Noise cleaning in FDD*).* For large input precision P, residual noise may displace the selector entries from the exact $\{0,1\}$ values required by the subsequent KProduct step. To address this, [25] insert a cleaning step Clean into the FDD step that evaluates the binary identity gate $G_{\mathsf{id}}(X) = 3X^2 - 2X^3$ on each ciphertext. This polynomial has a horizontal tangent at 0 and 1, so its Taylor expansion maps an input $b + \varepsilon$ with $b \in \{0,1\}$ to $b + O(\varepsilon^2)$, squaring the error magnitude at the cost of two additional multiplicative levels. In our implementation, we enable this cleaning for $P \ge 2^{14}$ to ensure correct decryption.

Complexity. The cost of LRMT-FBT comes from the three stages of Algorithm 3.

- FDD. This stage performs $\lceil \log_p P \rceil$ calls to SlotsMVB on plaintext space $\mathbb{Z}_p$, as well as negligible RLWE scheme operations, such as subtraction and modulus switching. For large P, it also evaluates the cleaning polynomial G_{id} on the Kronecker-delta vectors to reduce noise. We refer to Appendix C.2 for the detailed complexity.
- *Kronecker product.* The two KProduct calls compute the row and column selectors and perform $O(\sqrt{P})$ ciphertext multiplications as shown in Appendix C.3.
- *LUT evaluation.* The rank-r evaluation performs $2r\sqrt{P}$ scalar multiplications and r ciphertext multiplications.

In CMT-FBT, the LUT evaluation step requires P scalar multiplications, which grow with the table size and dominate the runtime at high precision.

By contrast, our low-rank evaluation reduces this complexity to $O(r\sqrt{P})$. For a smaller rank, this step is no longer the bottleneck, and the first two stages become dominant. The per-stage timing is present in Section 5.

Noise growth of Algorithm 3. The correctness of LRMT-FBT follows from the correctness of FDD, KProduct, LRMTEval, and the underlying CKKS operations. The output noise accumulates along the three stages of the pipeline.

- The FDD stage produces each Kronecker-delta entry with noise bounded by the bootstrapping noise B_{boot}, described in Appendix C.2. For large P, the optional Clean step squares the residual error of each entry before it is used.
- The two KProduct only multiply ciphertexts encrypting binary values, so the noise accumulation is additive rather than multiplicative. As demonstrated by CMT (Appendix C.3), the noise growth is linear in the number of inputs to each tree, which is $O(\log P)$, so each selector carries noise $B_{\mathsf{in}} = O(\log P) \cdot B_{\mathsf{boot}}$.
- Substituting $B_{\mathsf{in}} = O(\log P) \cdot B_{\mathsf{boot}}$ into the LRMTEval bound from Section 4.1 scales the B_{in} term by a factor of $O(\log P)$, which yields the output noise bound $|\varepsilon(\mathsf{ct}_{\mathrm{out}})|$.

Combining this with the deterministic truncation error $\sigma_{r+1}(\mathbf{M}_f)$ from Section 4.1, the total deviation between the decrypted output and $f(x)$ is bounded by

$$|f(x) - \mathsf{Dec}(\mathsf{ct}_{\mathrm{out}})| \;\le\; \sigma_{r+1}(\mathbf{M}_f) + |\varepsilon(\mathsf{ct}_{\mathrm{out}})|.$$

4.3 Extensions

The scheme of Section 4.2 evaluates a single LUT on one encrypted input. We now describe three extensions of the low-rank evaluation of Algorithm 2, i.e., several LUTs on a shared input, multi-input LUT, and a different LUT per SIMD slot.

Multi-value LUTs. When K LUTs $f^{(1)}, \ldots, f^{(K)}$ are evaluated on the same encrypted input, the FDD stage and the two KProduct calls are performed once and produce the selectors $\boldsymbol{\delta}_{\mathrm{MSD}}$ and $\boldsymbol{\delta}_{\mathrm{LSD}}$ shared by all K outputs as shown in Section 2.4. For each LUT $f^{(m)}$, we precompute an independent rank-r_m factorization

$$\mathbf{M}_{f^{(m)}}^{(r_m)} = \sum_{k=1}^{r_m} \sigma_k^{(m)}\, \mathbf{u}_k^{(m)} \big(\mathbf{v}_k^{(m)}\big)^{\top}$$

of its LUT matrix. Reusing the shared selectors, the total complexity is $\sum_{m=1}^{K} r_m(\alpha+\beta)$ scalar multiplication and $\sum_{m=1}^{k} r_m$ ciphertext multiplications, followed by the standard SlotsToCoeffs and modulus switch.

Multi-input LUTs. The low-rank evaluation extends naturally to multi-input LUTs when the radix-p digits are already available, eliminating the need for the FDD stage. For each encrypted digit $x_\nu \in \mathbb{Z}_p$, a single MVB call evaluates all

p coordinate maps $\delta_{x_\nu,a}$ (for $a \in \{0,\ldots,p-1\}$) and returns the corresponding Kronecker-delta vector $\mathbf{e}_{x_\nu} \in \{0,1\}^p$. Two subsequent KProduct calls then combine these vectors into the row and column selectors $\boldsymbol{\delta}_{\mathrm{MSD}}$ and $\boldsymbol{\delta}_{\mathrm{LSD}}$, following the same procedure as in the single-input pipeline. After reshaping the multi-input LUT into a matrix $\mathbf{M}_f \in \mathbb{R}^{\alpha\times\beta}$ under the same digit partition, Algorithm 2 applies without modification.

Slot-independent LUTs. LRMT-FBT can also support evaluating distinct LUTs f_h across different SIMD slots. Each f_h is factorized independently, and for a common rank bound r, the precomputation packs the singular-vector entries of each slot into CKKS plaintext such that slot h carries the factors of f_h. The evaluation procedure in Algorithm 2 remains unchanged, except that scalar multiplications are replaced by plaintext ciphertext multiplications. This substitution incurs higher computational cost and noise growth compared to scalar multiplication, representing the primary trade-off of this extension.

5 Experimental Evaluation

In this section, we primarily compare LRMT-FBT against the state-of-the-art CMT-FBT [25]. We first describe the experimental setup and the CKKS parameter sets and present the implementation optimizations. Then, we measure the LUT evaluation stage over a range of ranks, and show the bootstrapping performance through a stage-by-stage runtime breakdown.

5.1 Experimental Setup

All experiments were run on a server with two AMD EPYC 9474F 48-core CPUs at 4.1 GHz, 1 TiB of RAM, and Ubuntu 22.04.5 LTS. We implement LRMT-FBT in OpenFHE v1.2.0 [1], compiled with `clang++-14`, and pack N values into the RLWE ciphertext represented by the initial coefficients. Unless stated otherwise, all timings are measured on a single thread. Following CMT-FBT, we evaluate the LUT precisions $\log_2 P \in \{12, 14, 16, 18, 20\}$. Table 4 lists the CKKS parameter sets used in experiments, all of which provide 128-bit security. We use the sparse secret distribution with Hamming weight $h = 192$, and check the 128-bit security level of every parameter set with the lattice estimator.[4]

5.2 Implementation Optimizations

We further apply several implementation-level optimizations to accelerate LRMT-FBT (Algorithm 3). We describe the core techniques as follows, and show more details in Appendix E.

Lazy relinearization in KProduct. The KProduct builds a selector through a binary tree of ciphertext multiplications (Algorithm 6). At intermediate levels,

[4] `https://github.com/malb/lattice-estimator`

Table 4. CKKS parameter sets under 128-bit security.

Scheme	P	$\log_2 p$	$\mathbf{M}_f$	N	L	$\log_2 \Delta$	d_{num}	$\log_2 Q_L P$
CMT-FBT	2^{12}	3	64×64	2^{16}	22	48	4	1404
	2^{14}	4	256×64	2^{16}	23	48	4	1452
	2^{16}	4	256×256	2^{16}	23	48	4	1452
	2^{18}	4	2048×64	2^{16}	24	48	5	1500
	2^{20}	4	1024×1024	2^{17}	25	59	3	2074
LRMT-FBT	2^{12}	3	64×64	2^{16}	23	48	4	1452
	2^{14}	4	64×256	2^{16}	24	48	5	1500
	2^{16}	4	256×256	2^{16}	24	48	5	1500
	2^{18}	4	256×1024	2^{16}	25	48	5	1548
	2^{20}	5	1024×1024	2^{17}	26	59	3	2133

each product serves as input to a subsequent multiplication and must be relinearized to bound the ciphertext dimension. We note that the $O(\sqrt{P})$ products at the final level can be used directly in LRMTEval as operands for linear combinations with singular vectors, which preserve the ciphertext dimension. This allows us to defer relinearization until after these combinations, applying it to the r rank-one outputs of LRMTEval instead of the $O(\sqrt{P})$ selector products. Note that in CMT-FBT, one selector is multiplied directly by the dense LUT matrix and must be fully relinearized beforehand, so only the other side can be deferred. In our LRMT-FBT, neither selector is multiplied by a dense matrix, as both are combined with the singular vectors through scalar multiplications, so the relinearization is deferred on both sides.

Lazy rescale in LRMTEval. Recall that singular vectors are real-valued in Algorithm 2, and the linear combinations involve only real-scalar multiplications. A naive implementation rescales the ciphertext after every multiplication, incurring m rescale operations for a length-m linear combination. Since all terms in a linear combination share the same scale parameters (level), we delay rescale operation until after summation. This reduces the number of rescale operations per singular-vector inner product from m to one and prevents the accumulation of intermediate rounding errors. For balanced reshaping with $m = \sqrt{P}$, the optimization eliminates $\sqrt{P} - 1$ operations.

Choice of decomposition basis. Following the analysis of [25], we select the digit base p such that the number of digits $\lceil \log_p P \rceil$ is a power of two. This enables an even partition into two halves with $\alpha = \beta = \sqrt{P}$, balancing the computation across KProduct calls for $\boldsymbol{\delta}_{\text{MSD}}, \boldsymbol{\delta}_{\text{LSD}}$. Then, each Kronecker product is evaluated via a binary tree to minimize both ciphertext multiplications and multiplicative depth. In addition, a larger base increases the polynomial degree in the FDD stage but reduces the number of SlotsMVB calls, and we select p to optimize the trade-off between these two costs. For example, we set $p = 2^5$ at $P = 2^{20}$, which gives four digits and selectors of size $\alpha = \beta = 2^{10}$.

5.3 Performance of LUT Evaluation

We first compare LRMTEval against the dense matrix evaluation of CMT-FBT at $P = 2^{20}$ under the balanced factorization $\alpha = \beta = \sqrt{P}$. Table 5 present the runtime across ranks $r \in [1, 256]$. Among the Class A functions, step, sign, exp, modular reduction, and floor division have $r = 1$, ReLU, absolute value, and sin / cos have $r = 2$, and x^2 has $r = 3$. At these ranks LRMTEval is 196.9×, 99.7×, and 66.7× faster, respectively. For Class B functions Sigmoid, tanh, GeLU, and SiLU with numerical rank $r = 5$, LRMTEval achieves a 40.5× speedup. The runtime of LRMTEval grows linearly with r, whereas the method of CMT performs P scalar multiplications for all functions.

Table 5. LUT evaluation time of LRMT-FBT and CMT-FBT at $P = 2^{20}$ ($\sqrt{P} = 1024$). Speedup is $T_{\mathsf{CMT}}/T_{\mathsf{LRMT}}$.

Rank r	LRMT-FBT (s)	CMT-FBT (s)	**Speedup**
1	21.58	4,248.34	196.86×
2	42.62	4,248.34	99.68×
3	63.70	4,248.34	66.69×
4	93.14	4,248.34	45.61×
5	105.00	4,248.34	40.46×
8	160.52	4,248.34	26.47×
16	318.99	4,248.34	13.32×
32	650.94	4,248.34	6.53×
64	1,392.77	4,248.34	3.05×
128	3,854.93	4,248.34	1.10×
256	8,755.79	4,248.34	0.49×

Furthermore, at $r = 1$, LRMTEval requires only $2\sqrt{P} = 2048$ scalar multiplications compared to $P = 2^{20}$ for method of CMT, yielding a theoretical speedup of $\sqrt{P}/2 = 512$. The measured speedup of 196.9× is lower because each scalar multiplication in LRMTEval operates on real-valued singular-vector entries and must be followed by a costly rescale operation, whereas the method of CMT only involves integer entries that require no rescaling. Note that the two schemes achieve comparable performance only when $r \approx 128$. Although each multiplication incurs additional cost, all practically relevant Class A and Class B functions satisfy $r \leq 5$ as shown in Section 3, ensuring that LRMTEval outperforms dense matrix evaluation with in CMT. Finally, Table Table 7 shows a breakdown of the total runtime of the bootstrapping across different ranks.

5.4 Bootstrapping Performance

To analyze the bootstrapping performance breakdown, we split the functional bootstrapping of Algorithm 3 into five stages by treating noise cleaning (Re-

Table 6. Stage-by-stage runtime breakdown for ReLU ($r = 2$), single-threaded. All times in seconds. Parenthesized values are the per-stage speedup $T_{\mathsf{CMT}}/T_{\mathsf{LRMT}}$. The amortized time is defined as latency divided by the number of values N packed per ciphertext.

	$P = 2^{12}$		$P = 2^{14}$		$P = 2^{16}$		$P = 2^{18}$		$P = 2^{20}$	
Stage	CMT	LRMT	CMT	LRMT	CMT	LRMT	CMT	LRMT	CMT	LRMT
S1: FDD	108.29	107.48 (1.01×)	177.95	165.79 (1.07×)	190.54	176.64 (1.08×)	245.85	223.48 (1.10×)	539.36	645.05 (0.84×)
S2: Noise cleaning	–	–	16.78	20.73 (0.81×)	21.00	25.89 (0.81×)	25.66	28.63 (0.90×)	63.89	112.58 (0.57×)
S3: Kronecker product	13.33	8.79 (1.52×)	39.29	18.33 (2.14×)	48.32	29.46 (1.64×)	254.38	136.89 (1.86×)	641.48	259.22 (2.47×)
S4: LUT evaluation	**10.50**	**2.12 (4.95×)**	**31.56**	**3.49 (9.04×)**	**127.90**	**5.25 (24.36×)**	**543.73**	**13.02 (41.76×)**	**4248.34**	**42.62 (99.68×)**
S5: SlotsToCoeffs	1.34	1.35 (0.99×)	3.33	3.40 (0.98×)	3.32	3.39 (0.98×)	3.36	3.33 (1.01×)	24.47	13.65 (1.79×)
Latency	**133.79**	**119.91 (1.12×)**	**269.28**	**211.80 (1.27×)**	**391.58**	**240.71 (1.63×)**	**1073.25**	**405.98 (2.64×)**	**5524.22**	**1076.39 (5.13×)**
Amortized time (ms)	**2.04**	**1.83 (1.12×)**	**4.11**	**3.23 (1.27×)**	**5.98**	**3.67 (1.63×)**	**16.38**	**6.19 (2.64×)**	**42.15**	**8.21 (5.13×)**

mark 2) and SlotsToCoeffs separately from the three core stages (FDD, KProduct, LRMTEval) in Section 4.1. We show the latency and amortized time of each stage for ReLU ($r = 2$) across five LUT precisions in Table 6.

Table 7. Stage-by-stage runtime breakdown for LRMT-FBT at $P = 2^{20}$, $p = 2^5$ with different ranks r. All stage and latency times are in seconds. The amortized time is defined as latency divided by the number of packed values $N = 2^{17}$. The speedup is computed against CMT-FBT.

Stage	$r = 1$	$r = 2$	$r = 3$	$r = 4$	$r = 5$	$r = 8$	$r = 16$	$r = 32$	$r = 64$	$r = 128$	$r = 256$
S1: FDD	631.84	645.05	647.72	632.83	634.97	635.81	635.04	635.52	635.16	635.76	635.25
S2: Noise cleaning	113.63	112.58	116.72	115.75	112.98	114.68	114.05	114.47	114.13	114.63	114.24
S3: Kronecker product	259.41	259.22	258.79	260.03	260.27	259.84	259.29	259.66	259.36	259.78	259.46
S4: LUT evaluation	21.58	42.62	63.70	93.14	105.00	160.52	318.99	650.94	1,392.77	3,854.93	8,755.79
S5: SlotsToCoeffs	13.81	13.65	14.18	14.52	13.70	14.05	13.91	14.01	13.92	14.04	13.95
Latency	1,042.04	1,076.39	1,104.40	1,118.04	1,130.27	1,187.65	1,343.94	1,677.32	2,418.01	4,881.88	9,781.38
Amortized time (ms)	7.95	8.21	8.43	8.53	8.62	9.06	10.25	12.80	18.45	37.25	74.63
Speedup	**5.30×**	**5.13×**	**5.00×**	**4.94×**	**4.89×**	**4.65×**	**4.11×**	**3.29×**	**2.28×**	**1.13×**	0.56×

In CMT-FBT, the LUT evaluation stage accounts for 8% of the total latency at $P = 2^{12}$ and grows to 77% at $P = 2^{20}$. Since this stage dominates the computation at high precision, replacing it with LRMTEval directly determines the cost of the complete bootstrapping. As P increases from 2^{12} to 2^{20}, the bootstrapping speedup grows from 1.12× to 5.13×, and the total latency of ReLU drops from 5,524 s to 1,076 s. Table 1 reports the bootstrapping speedup of all benchmark functions at $P = 2^{20}$, which achieves 5.3× for the rank-1 Class A functions and 4.9× for the rank-5 Class B activations as indicated in Table 1. The remaining stages differ only slightly between the two schemes. In the FDD stage, we apply a lazy-conjugation technique that lowers the scalar-multiplication overhead, yielding a modest performance gain in Appendix E. In the KProduct stage, LRMT-FBT is 1.52× to 2.47× faster, since the selectors enter LRMTEval through scalar multiplications and the lazy relinearization (Section 5.2) is deferred on both the MSD and LSD sides. The noise cleaning and SlotsToCoeffs stages have comparable runtimes in both schemes.

We further note that both CMT-FBT and LRMT-FBT evaluate LUTs beyond the 16-bit limit of the direct AKP-FBT [2,25]. At $P = 2^{14}$, AKP-FBT requires 2,479 s under the same setting, whereas LRMT-FBT completes in 211.8 s, an 11.7× speedup. Finally, the full per-stage breakdown, the multi-threaded results, and the bootstrapping key sizes are reported in Section E.

6 Conclusion

In this paper, we present LRMT-FBT which improves CKKS functional bootstrapping by replacing the dense LUT evaluation with a rank-r truncated singular value decomposition. For the LUT size P and under balanced factorization, our method reduces scalar multiplications from P to $2r\sqrt{P}$ and ciphertext multiplications from $\sqrt{P}$ to r. To enable practical rank selection, we introduce a spectral taxonomy framework that classifies LUT matrices by their singular-value decay. Class A functions possess exact low rank independent of P, Class B activations achieve numerical rank $r = O(\log P)$, and Class C functions admit no compression. We implement LRMT-FBT in OpenFHE and evaluate it at different precisions. Typically, for the ReLU function (rank = 2) at $P = 2^{20}$, our method accelerates the LUT evaluation stage by up to 196.9× and the bootstrapping by up to 5.3× over CMT-FBT. By turning the spectral structure of the lookup table into a concrete efficiency gain, LRMT-FBT makes high-precision functional bootstrapping in CKKS substantially more practical.

References

1. Al Badawi, A., Bates, J., Bergamaschi, F., Cousins, D.B., Erabelli, S., Genise, N., Halevi, S., Hunt, H., Kim, A., Lee, Y., et al.: Openfhe: Open-source fully homomorphic encryption library (2022)
2. Alexandru, A., Kim, A., Polyakov, Y.: General functional bootstrapping using ckks. In: Annual International Cryptology Conference. pp. 304–337. Springer (2025)
3. Bae, Y., Cheon, J.H., Kim, J., Stehlé, D.: Bootstrapping bits with ckks. In: Annual International Conference on the Theory and Applications of Cryptographic Techniques. pp. 94–123. Springer (2024)
4. Bae, Y., Kim, J., Stehlé, D., Suvanto, E.: Bootstrapping small integers with ckks. In: International Conference on the Theory and Application of Cryptology and Information Security. pp. 330–360. Springer (2024)
5. Beckermann, B., Townsend, A.: On the singular values of matrices with displacement structure. SIAM Journal on Matrix Analysis and Applications **38**(4), 1227–1248 (2017)
6. Bergerat, L., Boudi, A., Bourgerie, Q., Chillotti, I., Ligier, D., Orfila, J.B., Tap, S.: Parameter optimization and larger precision for (t) fhe: L. bergerat et al. Journal of Cryptology **36**(3), 28 (2023)
7. Bergerat, L., Orfila, J.B., Roux-Langlois, A., Tap, S.: Accelerating tfhe with sorted bootstrapping techniques. In: International Conference on the Theory and Application of Cryptology and Information Security. pp. 68–100. Springer (2025)

8. Bian, S., Fu, Y., Shen, R., Pan, H., Wang, A., Guan, Z.: High-precision functional bootstrapping for ckks from fourier extension. In: Annual International Conference on the Theory and Applications of Cryptographic Techniques. pp. 274–303. Springer (2026)
9. Bossuat, J.P., Troncoso-Pastoriza, J., Hubaux, J.P.: Bootstrapping for approximate homomorphic encryption with negligible failure-probability by using sparse-secret encapsulation. In: International Conference on Applied Cryptography and Network Security. pp. 521–541. Springer (2022)
10. Brakerski, Z., Gentry, C., Vaikuntanathan, V.: (leveled) fully homomorphic encryption without bootstrapping. ACM Transactions on Computation Theory (TOCT) **6**(3), 1–36 (2014)
11. Carpov, S., Izabachène, M., Mollimard, V.: New techniques for multi-value input homomorphic evaluation and applications. In: Cryptographers' Track at the RSA Conference. pp. 106–126. Springer (2019)
12. Cheon, J.H., Han, K., Kim, A., Kim, M., Song, Y.: A full rns variant of approximate homomorphic encryption. In: International Conference on Selected Areas in Cryptography. pp. 347–368. Springer (2018)
13. Cheon, J.H., Hanrot, G., Kim, J., Stehlé, D.: Ship: A shallow and highly parallelizable ckks bootstrapping algorithm. In: Annual International Conference on the Theory and Applications of Cryptographic Techniques. pp. 398–428. Springer (2025)
14. Cheon, J.H., Kim, A., Kim, M., Song, Y.: Homomorphic encryption for arithmetic of approximate numbers. In: International conference on the theory and application of cryptology and information security. pp. 409–437. Springer (2017)
15. Cheon, J.H., Kim, D., Kim, D.: Efficient homomorphic comparison methods with optimal complexity. In: International Conference on the Theory and Application of Cryptology and Information Security. pp. 221–256. Springer (2020)
16. Cheon, Jung Hee and Han, Kyoohyung and Kim, Andrey and Kim, Miran and Song, Yongsoo: Bootstrapping for approximate homomorphic encryption. In: Annual International Conference on the Theory and Applications of Cryptographic Techniques. pp. 360–384. Springer (2018)
17. Chillotti, I., Gama, N., Georgieva, M., Izabachene, M.: Faster fully homomorphic encryption: Bootstrapping in less than 0.1 seconds. In: international conference on the theory and application of cryptology and information security. pp. 3–33. Springer (2016)
18. Chillotti, I., Gama, N., Georgieva, M., Izabachène, M.: Faster packed homomorphic operations and efficient circuit bootstrapping for tfhe. In: International Conference on the Theory and Application of Cryptology and Information Security. pp. 377–408. Springer (2017)
19. Chillotti, I., Gama, N., Georgieva, M., Izabachène, M.: Tfhe: Fast fully homomorphic encryption over the torus: I. chillotti et al. Journal of Cryptology **33**(1), 34–91 (2020)
20. Chung, H., Kim, H., Kim, Y.S., Lee, Y.: Amortized large look-up table evaluation with multivariate polynomials for homomorphic encryption. Cryptology ePrint Archive (2024)
21. Clet, P.E., Boudguiga, A., Sirdey, R., Zuber, M.: Combo: a novel functional bootstrapping method for efficient evaluation of nonlinear functions in the encrypted domain. In: International Conference on Cryptology in Africa. pp. 317–343. Springer (2023)

22. De Micheli, G., Kim, D., Micciancio, D., Suhl, A.: Faster amortized fhew bootstrapping using ring automorphisms. In: IACR International Conference on Public-Key Cryptography. pp. 322–353. Springer (2024)
23. Drucker, N., Moshkowich, G., Pelleg, T., Shaul, H.: Bleach: Cleaning errors in discrete computations over ckks: N. drucker et al. Journal of Cryptology **37**(1), 3 (2024)
24. Ducas, L., Micciancio, D.: Fhew: bootstrapping homomorphic encryption in less than a second. In: Annual international conference on the theory and applications of cryptographic techniques. pp. 617–640. Springer (2015)
25. Dumezy, J., Alexandru, A., Polyakov, Y., Clet, P.E., Chakraborty, O., Boudguiga, A.: Evaluating larger lookup tables using ckks. Cryptology ePrint Archive (2025)
26. Fan, J., Vercauteren, F.: Somewhat practical fully homomorphic encryption. Cryptology ePrint Archive (2012)
27. Gentry, C.: Fully homomorphic encryption using ideal lattices. In: Proceedings of the forty-first annual ACM symposium on Theory of computing. pp. 169–178 (2009)
28. Golub, G.H., Van Loan, C.F.: Matrix computations. JHU press (2013)
29. Guimarães, A., Borin, E., Aranha, D.F.: Revisiting the functional bootstrap in tfhe. IACR Transactions on Cryptographic Hardware and Embedded Systems pp. 229–253 (2021)
30. Guimarães, A., Pereira, H.V.: Fast amortized bootstrapping with small keys and polynomial noise overhead. In: Proceedings of the 2025 ACM SIGSAC Conference on Computer and Communications Security. pp. 2967–2981 (2025)
31. Guimarães, A., Pereira, H.V., Van Leeuwen, B.: Amortized bootstrapping revisited: Simpler, asymptotically-faster, implemented. In: International Conference on the Theory and Application of Cryptology and Information Security. pp. 3–35. Springer (2023)
32. Horn, R.A., Johnson, C.R.: Matrix analysis. Cambridge university press (2012)
33. Lee, E., Lee, J.W., No, J.S., Kim, Y.S.: Minimax approximation of sign function by composite polynomial for homomorphic comparison. IEEE Transactions on Dependable and Secure Computing **19**(6), 3711–3727 (2021)
34. Liu, Z., Wang, Y.: Amortized functional bootstrapping in less than 7 ms, with o˜(1) polynomial multiplications. In: International Conference on the Theory and Application of Cryptology and Information Security. pp. 101–132. Springer (2023)
35. Paterson, M.S., Stockmeyer, L.J.: On the number of nonscalar multiplications necessary to evaluate polynomials. SIAM Journal on Computing **2**(1), 60–66 (1973)
36. Trefethen, L.N.: Approximation theory and approximation practice, extended edition. SIAM (2019)
37. Wang, R., Ha, J., Shen, X., Lu, X., Chen, C., Wang, K., Lee, J.: Refined tfhe leveled homomorphic evaluation and its application. In: Proceedings of the 2025 ACM SIGSAC Conference on Computer and Communications Security. pp. 2309–2323 (2025)
38. Z. Liu and Y. Wang: Relaxed functional bootstrapping: A new perspective on bgv. Cryptology ePrint Archive, Paper 2024/172 (2024)
39. Zama, A.: Tfhe-rs: A pure rust implementation of the tfhe scheme for boolean and integer arithmetics over encrypted data (2022)

A RNS CKKS Operations

We introduce the operations on RNS CKKS ciphertexts that we used in our implementation. We assume that the ciphertexts or plaintexts have the same

modulus. If the modulus Q'_ℓ differ, we first perform $\mathsf{DropLevels}$ to drop the ciphertext with the highest modulus to the same level as the ciphertext with the smallest modulus.

- $\mathsf{KeySwitchGen}(\mathbf{s}, \mathbf{s}_{\mathsf{new}})$: For the gadget modulus P' and a decomposition base w, both power of two, let $d_{\mathsf{swk}} = \lceil \log_w(Q'_L) \rceil$. Sample uniformly at random $a'_j \leftarrow \mathcal{R}_{Q'_L P'}$ and errors $e'_j \leftarrow \chi_{\mathsf{err}}$ and return $\mathsf{swk} = \{(\mathsf{swk}_{j,0}, \mathsf{swk}_{j,1})\}_{j=0}^{d_{\mathsf{swk}}-1}$ with
$$\mathsf{swk}_{j,0} = -a'_j \cdot \mathbf{s}_{\mathsf{new}} + e'_j + P' w^j \cdot \mathbf{s}, \qquad \mathsf{swk}_{j,1} = a'_j.$$
Set $\mathsf{rlk} = \mathsf{KeySwitchGen}(\mathbf{s}, \mathbf{s}^2)$ and $\mathsf{cnk} = \mathsf{KeySwitchGen}(\mathbf{s}, \mathbf{s}^{-1})$.
- $\mathsf{KeySwitch}(\mathsf{ct}, \mathsf{swk})$: For $\mathsf{ct} = (c_0, c_1) \in \mathcal{R}^2_{Q'_\ell}$ and $\mathsf{swk} = \{(\mathsf{swk}_{j,0}, \mathsf{swk}_{j,1})\}_{j=0}^{d_{\mathsf{swk}}-1}$, decompose c_1 into d_{swk} polynomial digits $\{c_{1,j}\}_{j=0}^{d_{\mathsf{swk}}-1}$ so that $c_1 = \sum_{j=0}^{d_{\mathsf{swk}}-1} w^j \cdot c_{1,j}$. Return
$$\left(\left\lfloor \frac{c_0 + \sum_j c_{1,j} \cdot \mathsf{swk}_{j,0}}{P'} \right\rceil, \left\lfloor \frac{\sum_j c_{1,j} \cdot \mathsf{swk}_{j,1}}{P'} \right\rceil \right)_{Q'_\ell}.$$
- $\mathsf{DropLevels}(\{\mathsf{ct}, Q'_\ell\}, k \le \ell)$: return $\{\mathsf{ct}, Q'_{\ell-k}\}$.
- $\mathsf{AddConst}(\{\mathsf{ct}, Q'_\ell\}, z)$: For $\mathsf{ct} \in \mathcal{R}^2_{Q'_\ell}$ with scaling factor Δ and $z = a + ib \in \mathbb{C}$, set $\mathsf{ct}_{\mathsf{Add}} \leftarrow [\mathsf{ct} + (\lfloor \Delta \cdot (a + b \cdot X^{N/2}) \rceil, 0)]_{Q'_\ell}$ and return $\{\mathsf{ct}_{\mathsf{Add}}, Q'_\ell\}$.
- $\mathsf{AddPlain}(\{\mathsf{ct}, Q'_\ell\}, \mathsf{pt})$: For $\mathsf{ct} \in \mathcal{R}^2_{Q'_\ell}$ and $\mathsf{pt} \in \mathcal{R}$, set $\mathsf{ct}_{\mathsf{Add}} \leftarrow [\mathsf{ct} + (\mathsf{pt}, 0)]_{Q'_\ell}$ and return $\{\mathsf{ct}_{\mathsf{Add}}, Q'_\ell\}$.
- $\mathsf{Add}(\{\mathsf{ct}_1, Q'_\ell\}, \{\mathsf{ct}_2, Q'_\ell\})$: For $\mathsf{ct}_1, \mathsf{ct}_2 \in \mathcal{R}^2_{Q'_\ell}$, set $\mathsf{ct}_{\mathsf{Add}} \leftarrow [\mathsf{ct}_1 + \mathsf{ct}_2]_{Q'_\ell}$ and return $\{\mathsf{ct}_{\mathsf{Add}}, Q'_\ell\}$.
- $\mathsf{MulScal}(\{\mathsf{ct}, Q'_\ell\}, z, \Delta')$: For $\mathsf{ct} \in \mathcal{R}^2_{Q'_\ell}$ with scaling factor Δ and $z = a + ib \in \mathbb{C}$, set $\mathsf{ct}_{\mathsf{Mul}} \leftarrow \lfloor \Delta' a \rceil \cdot \mathsf{ct} + \lfloor \Delta' b \rceil \cdot \mathsf{ct} \cdot X^{N/2}$ and return $\{\mathsf{ct}_{\mathsf{Mul}}, Q'_\ell\}$ with a new scaling factor $\Delta\Delta'$.
- $\mathsf{MulPlain}(\{\mathsf{ct}, Q'_\ell\}, \mathsf{pt})$: For $\mathsf{ct} = (c_0, c_1) \in \mathcal{R}^2_{Q'_\ell}$ with scaling factor Δ and $\mathsf{pt} \in \mathcal{R}$ with scaling factor Δ', set $\mathsf{ct}_{\mathsf{Mul}} \leftarrow (c_0 \cdot \mathsf{pt}, c_1 \cdot \mathsf{pt})$ and return $\{\mathsf{ct}_{\mathsf{Mul}}, Q'_\ell\}$ with a new scaling factor $\Delta\Delta'$.
- $\mathsf{Mul}(\{\mathsf{ct}_1, Q'_\ell\}, \{\mathsf{ct}_2, Q'_\ell\})$: For $\mathsf{ct}_i = (b_i, a_i) \in \mathcal{R}^2_{Q'_\ell}$ with scaling factor Δ_i, compute $(d_0, d_1, d_2) = [(b_1 b_2, a_1 b_2 + a_2 b_1, a_1 a_2)]_{Q'_\ell}$, set $\mathsf{ct}_{\mathsf{Mul}} \leftarrow [(d_0, d_1) + \mathsf{KeySwitch}(d_2, \mathsf{rlk})]_{Q'_\ell}$ and return $\{\mathsf{ct}_{\mathsf{Mul}}, Q'_\ell\}$ with a new scaling factor $\Delta_1\Delta_2$.
- $\mathsf{Rescale}(\{\mathsf{ct}, Q'_\ell\})$: For $\mathsf{ct} \in \mathcal{R}^2_{Q'_\ell}$ with scaling factor Δ, set $\mathsf{ct}_{\mathsf{Rscl}} \leftarrow \lfloor q'^{-1}_\ell \cdot \mathsf{ct} \rceil$ with $q'^{-1}_\ell \in \mathcal{R}$ and return $\{\mathsf{ct}_{\mathsf{Rscl}}, Q'_{\ell-1}\}$ with a new scaling factor Δ / q'_ℓ.
- $\mathsf{Conj}(\{\mathsf{ct}, Q'_\ell\})$: For $\mathsf{ct} = (c_0, c_1) \in \mathcal{R}^2_{Q'_\ell}$, set
$$\mathsf{ct}_{\mathsf{Conj}} \leftarrow [(c_0^{-1}, 0) + \mathsf{KeySwitch}(c_1, \mathsf{cnk})]_{Q'_\ell}.$$

B Paterson–Stockmeyer Algorithm

The Paterson–Stockmeyer (PS) algorithm [35] is a classical method for reducing the multiplicative complexity of univariate polynomial evaluation. Through a

baby-step giant-step strategy, it reduces the number of ciphertext multiplications for a degree-p polynomial from $O(p)$ to $O(\sqrt{p})$, as summarized in Algorithm 4. This reduction is important in homomorphic evaluation, where a ciphertext multiplication is far more expensive than an addition or a scalar multiplication.

Algorithm 4: Paterson–Stockmeyer polynomial evaluation.

procedure PolyPS($\{a_i\}_{0\le i\le p}, x$)
  Choose k, m such that $k \approx \sqrt{p/2}$ and $k(2^m - 1) > p$
  $\widetilde{P}(X) \leftarrow P(X) + X^{k(2^m-1)}$ // $P(X) = \sum_{i=0}^{p} a_i X^i$
  Precompute baby steps $(x, x^2, \ldots, x^{k-1})$ and giant steps $(x^k, x^{2k}, \ldots, x^{2^{m-1}k})$
  $X^{2^{m-1}k} q(X) + r(X) \leftarrow \widetilde{P}(X)$ // long division
  $\widetilde{r}(X) \leftarrow r(X) - X^{(2^{m-1}-1)k}$
  $c(X)q(X) + s(X) \leftarrow \widetilde{r}(X)$ // long division
  Evaluate $c(x)$ from the baby steps
  $\widetilde{s}(X) \leftarrow s(X) + X^{k(2^{m-1}-1)}$
  Recursively evaluate $q(x)$ and $\widetilde{s}(x)$
  $\widetilde{P}(x) \leftarrow \big(x^{2^{m-1}k} + c(x)\big)q(x) + \widetilde{s}(x)$
  $x^{k(2^m-1)} \leftarrow$ product of the giant steps
  return $P(x) = \widetilde{P}(x) - x^{k(2^m-1)}$

C FHE Algorithms

C.1 Multi-Value Bootstrapping

We give the full procedure of MVB (Section 2.4) in Algorithm 5, together with complexity and noise growth.

Complexity. The multiplicative depth is the same as AKP-FBT, and all steps up to EvalExp are computed once and shared across the k LUTs. Only the per-LUT evaluation of a degree-$(p-1)$ Hermite interpolant by the Paterson–Stockmeyer algorithm, together with the final SlotsToCoeffs and ModSwitch, is repeated k times. We refer to [25] for the detailed cost.

Noise. The noise behavior follows directly from the analysis of AKP-FBT. Let $x + \eta$ denote the slot value after the shared stages, where η is the normalized input noise. For an order-M Hermite interpolation, all derivatives up to order M are zero at the interpolation points. Consequently, whenever $|\eta| < 1/(2p)$, the interpolation error satisfies $|R(x+\eta) - R(x)| \le C|\eta|^{M+1}$ for some constant C. Hence, the interpolation reduces the input noise at order $M+1$. In particular, the first-order Hermite interpolation used in AKP-FBT and MVB provides quadratic noise reduction. Let $\varepsilon_{\mathrm{AKP}}$ denote an upper bound on the noise in the output ciphertext after bootstrapping, accounting for both the Hermite-interpolation

Algorithm 5: Multi-value bootstrapping and its slot-encoded variant.

procedure $\mathsf{MVB}\big(\mathsf{ct} \in \mathcal{R}_q^2, \{\mathsf{LUT}(f_i)\}_{1\le i\le k}, p, k\big)$
  $\mathsf{ct}_1 \leftarrow \mathsf{ModSwitch}(\mathsf{ct}, q_0')$
  $\mathsf{ct}_2 \leftarrow \mathsf{ModRaise}(\Delta \cdot \mathsf{ct}_1/q_0', Q_L')$
  $\mathsf{ct}_3 \leftarrow \mathsf{CoeffsToSlots}(\mathsf{ct}_2)$
  $\mathsf{ct}_{\exp} \leftarrow \mathsf{EvalExp}(\mathsf{ct}_3)$ // Approximation of $x \mapsto \exp(2\pi \mathrm{i} x/p)$
  $\{\mathsf{ct}_{\mathrm{pow},j}\}_{j\in I_{\mathrm{EvalLUT}}} \leftarrow \mathsf{EvalPower}(\mathsf{ct}_{\exp}, p)$
  for $i \in \{1,\dots,k\}$ **do**
    $\mathsf{ct}_{4,i} \leftarrow \mathsf{EvalLUT}(\{\mathsf{ct}_{\mathrm{pow},j}\}_{j\in I_{\mathrm{EvalLUT}}}, \mathsf{LUT}(f_i))$
    // Evaluate the Hermite interpolation of f_i using the precomputed powers and the Paterson-Stockmeyer algorithm
    $\mathsf{ct}_{5,i} \leftarrow \mathsf{MulScal}(\mathsf{Add}(\mathsf{ct}_{4,i}, \mathsf{Conj}(\mathsf{ct}_{4,i})), 0.5)$ // Compute the real part
  // Returning $\{\mathsf{ct}_{5,i}\}_{1\le i\le k}$ gives SlotsMVB
  for $i \in \{1,\dots,k\}$ **do**
    $\mathsf{ct}_{6,i} \leftarrow \mathsf{SlotsToCoeffs}(\mathsf{ct}_{5,i})$
    $\mathsf{ct}_{7,i} \leftarrow \mathsf{ModSwitch}(\mathsf{ct}_{6,i}, Q)$
  return $\{\mathsf{ct}_{7,i}\}_{1\le i\le k}$

error and the noise introduced by CKKS homomorphic evaluation. Since all LUT outputs differ only in the interpolation coefficients used in the final polynomial evaluation, every output ciphertext satisfies the same bound $\varepsilon_{\mathrm{AKP}}$.

C.2 Functional Digit Decomposition

We describe the complexity and noise growth of functional digit decomposition, whose procedure is given in Algorithm 1 of Section 2.4.

Complexity. FDD needs to perform $t = \lceil \log_p P \rceil$ calls to SlotsMVB, together with the per-iteration SlotsToCoeffs and ModSwitch of the digit-removal step. The cost is identical to that of [25], to which we refer for the detailed analysis.

Noise. The noise behavior of FDD follows from the noise analysis of MVB. In each iteration, the current digit is first processed by SlotsMVB. For the mod_p output, FDD further applies SlotsToCoeffs and ModSwitch before subtracting it from the current ciphertext. Together, these steps form the corresponding MVB procedure, so this coefficient-encoded output satisfies the same noise bound as MVB. Assume that the subtraction and ModSwitch operations in the loop introduce only negligible noise and preserve the input condition required by the next bootstrapping iteration. Thus, each MVB invocation brings the processed digit ciphertext back to the same noise bound, preventing noise accumulation across FDD iterations.

C.3 Kronecker Product

We give the full procedure of KProduct (Section 2.4) in Algorithm 6, together with its complexity and noise growth.

Algorithm 6: Kronecker products of CKKS ciphertext vectors.

procedure $\mathsf{KProduct}\Big(\{\mathsf{ctVec}_r\}_{1\le r\le n} \in (\mathcal{R}^2_{Q'_\ell})^{p_1} \times \cdots \times (\mathcal{R}^2_{Q'_\ell})^{p_n}\Big)$
  $n_0 \leftarrow n$
  for $i \in \{1, \ldots, \lceil \log n_0 \rceil\}$ **do**
    for $j \in \{1, \ldots, \lfloor n/2 \rfloor\}$ **do**
      $l_1 \leftarrow \text{length}(\mathsf{ctVec}_{2j-1}), \quad l_2 \leftarrow \text{length}(\mathsf{ctVec}_{2j})$
      for $k \in \{1, \ldots, l_1 l_2\}$ **do**
        $i_1 \leftarrow ((k-1) \bmod l_1) + 1$
        $i_2 \leftarrow \lfloor (k-1)/l_1 \rfloor + 1$
        $\mathsf{ctVec}_j[k] \leftarrow \mathsf{Mul}(\mathsf{ctVec}_{2j-1}[i_1], \mathsf{ctVec}_{2j}[i_2])$
    if $n \bmod 2 = 1$ **then**
      $\mathsf{ctVec}_{\lceil n/2 \rceil} \leftarrow \mathsf{ctVec}_n$
    $n \leftarrow \lceil n/2 \rceil$
  return $\mathsf{ctVec}_1 \in (\mathcal{R}^2_{Q'_{\ell - \lceil \log n_0 \rceil}})^{p_1 \cdots p_{n_0}}$

Complexity. KProduct evaluates the product as a balanced binary tree of $\lceil \log \sqrt{P} \rceil$ levels, so its multiplicative depth is logarithmic in the number of inputs. The final level performs $p_1 \cdots p_{\sqrt{P}}$ ciphertext multiplications and dominates the count. We refer to [25] for the detailed cost.

Noise. The inputs to KProduct are ciphertext vectors encrypting the outputs of Kronecker-delta functions. Since the output of a Kronecker-delta function is binary, every ciphertext in these vectors encrypts a value in $\{0, 1\}$. For two ciphertexts encrypting binary messages with input noises e_1 and e_2, assume that $e_1, e_2 \le |\varepsilon_{\text{in}}|$ and $e_1 e_2/\Delta < B_{\mathsf{err}}$. Then the noise after ciphertext multiplication is bounded by

$$e_1 + e_2 + \frac{e_1 e_2}{\Delta} + B_{\mathsf{err}} \le 2|\varepsilon_{\text{in}}| + 2B_{\mathsf{err}},$$

where $B_{\mathsf{err}} \in \mathbb{R}^+$ bounds the additional noise introduced by relinearization and rescaling. So the output noise of the product of n ciphertexts encrypting binary values is bounded by $n\big(|\varepsilon_{\text{in}}| + B_{\mathsf{err}}\big)$. Thus, the noise grows linearly with the number of input ciphertexts. Since KProduct takes a number of inputs that is logarithmic in the LUT size, the Kronecker-product noise grows logarithmically in the LUT size.

D Proof of Theorem 2

The proof strategy is to construct an auxiliary rank-r matrix $\widetilde{M}_r$ that approximates $\mathbf{M}_f$ entrywise within the Chebyshev truncation error of g, and then to bound $\sigma_{r+1}(\mathbf{M}_f)$ via the Eckart–Young theorem. Since g is analytic in a complex neighborhood of $[0, 1]$, there exists $\rho > 1$ such that the closure of the Bernstein ellipse $E_\rho^{[0,1]} \subset \mathbb{C}$, the ellipse with $\{0, 1\}$ and parameter ρ [36], is contained in this neighborhood. Let $M = \max_{z \in E_\rho^{[0,1]}} |g(z)|$. By the standard Chebyshev

truncation bound for functions analytic in a Bernstein ellipse [36], there exists a polynomial p_{r-1} of degree at most $r-1$ such that

$$\sup_{t\in[0,1]} |g(t) - p_{r-1}(t)| \le \frac{2M}{\rho-1}\rho^{-(r-1)}.$$

Expanding the degree-$(r-1)$ polynomial p_{r-1} in the monomial basis as $p_{r-1}(t) = \sum_{m=0}^{r-1} c_m t^m$, we obtain $p_{r-1}\left(\frac{i\beta+j}{P}\right) = \sum_{m=0}^{r-1} c_m \left(\frac{i\beta+j}{P}\right)^m$. For each m, the binomial expansion gives $(i\beta+j)^m = \sum_{a=0}^{m} \binom{m}{a}(i\beta)^{m-a} j^a$. Therefore,

$$p_{r-1}\left(\frac{i\beta+j}{P}\right) = \sum_{a=0}^{r-1} \phi_a(i)\psi_a(j),$$

where $\phi_a(i) = \sum_{m=a}^{r-1} c_m \binom{m}{a} \frac{(\beta i)^{m-a}}{P^m}$, $\psi_a(j) = j^a$. Define $\widetilde{M}_r \in \mathbb{R}^{\alpha\times\beta}$ by $\widetilde{M}_r[i,j] = p_{r-1}((i\beta+j)/P)$. Since each entry is a sum of r separable terms, Lemma 1 implies that $\operatorname{rank}(\widetilde{M}_r) \le r$.

For every $0 \le i < \alpha$ and $0 \le j < \beta$, we have $|\mathbf{M}_f[i,j] - \widetilde{M}_r[i,j]| \le \frac{2M}{\rho-1}\rho^{-(r-1)}$. Thus

$$\|\mathbf{M}_f - \widetilde{M}_r\|_{\max} \le \frac{2M}{\rho-1}\rho^{-(r-1)}.$$

Using $\|A\|_2 \le \|A\|_F \le \sqrt{\alpha\beta}\|A\|_{\max}$ for any $A \in \mathbb{R}^{\alpha\times\beta}$, we obtain

$$\|\mathbf{M}_f - \widetilde{M}_r\|_2 \le \frac{2M\sqrt{\alpha\beta}}{\rho-1}\rho^{-(r-1)}.$$

Finally, by the Eckart–Young theorem [28], the $(r+1)$-st singular value is the spectral-norm distance from $\mathbf{M}_f$ to the set of matrices of rank at most r,

$$\sigma_{r+1}(\mathbf{M}_f) = \min_{\operatorname{rank}(A)\le r} \|\mathbf{M}_f - A\|_2.$$

Since $\operatorname{rank}(\widetilde{M}_r) \le r$, we conclude

$$\sigma_{r+1}(\mathbf{M}_f) \le \|\mathbf{M}_f - \widetilde{M}_r\|_2 \le \frac{2M\sqrt{\alpha\beta}}{\rho-1}\rho^{-(r-1)}.$$

The factor $\sqrt{\alpha\beta}$ above comes from bounding the spectral norm by the Frobenius norm in $\|A\|_2 \le \|A\|_F \le \sqrt{\alpha\beta}\|A\|_{\max}$. It is conservative. For LUT matrices with displacement structure, a dimension-free bound on the singular values is available [5], which removes this factor. The looser form already suffices for the $O(\log P)$ rank conclusion drawn in Section 3.3.

E More Experimental Results

E.1 Performance of FDD

Paterson–Stockmeyer decomposition and lazy conjugation in the FDD stage. Each SlotsMVB call of the FDD stage (Algorithm 1) evaluates a mod-p

polynomial together with a set of Kronecker-delta polynomials, and we optimize both scalar multiplications and ciphertext multiplications. For the ciphertext multiplications, we parameterize the Paterson–Stockmeyer decomposition once for all p polynomials rather than separately for each. We precompute the powers $x, \ldots, x^{p-1}$ a single time, so that every polynomial is then evaluated with scalar multiplications and additions alone, which reduces the ciphertext multiplications per call from $O(p\sqrt{p})$ to $O(p)$. For the scalar multiplications, a complex coefficient $a + ib$ requires forming $a\mathsf{ct}$ and $b\mathsf{ct}$ followed by a conjugation to combine the real and imaginary parts, and each conjugation invokes a key-switching operation that is far more expensive than a real scalar multiplication [1]. We defer the conjugation across the entire linear combination, computing $\mathsf{ct}_{\mathrm{re}} = \sum_i a_i \mathsf{ct}_i$ and $\mathsf{ct}_{\mathrm{im}} = \sum_i b_i \mathsf{ct}_i$ with real scalar multiplications only and applying one conjugation at the end to form $\mathsf{ct}_{\mathrm{re}} + i\mathsf{ct}_{\mathrm{im}}$, reducing the number of conjugations per linear combination from p to one. Together, these two optimizations make the FDD stage of LRMT-FBT 1.01× to 1.10× faster than CMT-FBT for $P \leq 2^{18}$.

Table 8. Runtime and output level breakdown for CMT-FBT. Time and output level are reported in separate columns. Non-total rows report time in milliseconds, while Total rows report time in seconds. A dash indicates that no output level was reported in the log.

Module	Step	$P = 2^{12}$, $p = 2^3$		$P = 2^{14}$, $p = 2^4$		$P = 2^{16}$, $p = 2^4$		$P = 2^{18}$, $p = 2^4$		$P = 2^{20}$, $p = 2^4$	
		Time	**Level**	**Time**	**Level**	**Time**	**Level**	**Time**	**Level**	**Time**	**Level**
Input conversion	RLWE→CKKS (ms)	25.87	0	30.54	0	32.81	0	33.45	0	78.19	0
FDD	ModRaise (ms)	228.56	22	236.85	23	238.07	23	308.03	24	692.40	25
	CoeffToSlot (ms)	29,160.51	17	40,148.80	19	40,108.48	19	57,784.23	20	90,091.62	20
	ComputeExp (ms)	50,254.62	9	57,315.54	11	57,368.64	11	77,615.48	12	152,285.78	12
	Kronecker delta (ms)	24,189.47	5	61,704.71	6	74,346.97	6	85,504.17	7	205,731.67	7
	SlotToCoeff (ms)	3,981.00	0	18,024.18	0	17,960.10	0	23,965.71	0	89,059.07	0
	Total (s)	**108.29**	–	**177.95**	–	**190.54**	–	**245.85**	–	**539.36**	–
Cleaning	Cleaning (ms)	-	-	16,780.52	4	20,998.44	4	25,656.13	5	63,894.92	5
Multiplexer tree	Kronecker product (ms)	13,331.37	4	39,288.21	3	48,322.65	3	254,383.59	3	641,480.52	3
	LUT evaluation (ms)	10,500.66	3	31,562.81	2	127,898.00	2	543,731.69	2	4,248,336.82	2
	Total (s)	**24.10**	–	**71.16**	–	**176.65**	–	**798.31**	–	**4,896.35**	–
Output conversion	SlotToCoeff (ms)	1,336.78	0	3,328.75	0	3,323.38	0	3,362.67	0	24,474.35	0
	CKKS→RLWE (ms)	34.65	–	33.79	–	34.03	–	34.17	–	69.54	–
Overall	**Total (s)**	**133.79**	–	**269.28**	–	**391.58**	–	**1,073.25**	–	**5,524.22**	–

E.2 Multi-threaded Performance and Memory

LRMT-FBT retains high parallel scalability by exploiting independent components throughout the pipeline. We parallelize the per-digit PolyPS evaluations within each SlotsMVB of the FDD stage, the two KProduct trees, and the final block loop. Furthermore, the r rank-one terms in LRMTEval are mutually independent and can be evaluated in parallel without synchronization. Table 10 presents the performance, including the amortized time, with 8 threads on the same hardware. At $P = 2^{20}$, LRMT-FBT achieves a latency of 275.03 s and an amortized time of 2.10 ms.

Memory usage and key sizes. LRMT-FBT also reduces the plaintext storage of the LUT. Whereas CMT-FBT stores all P entries of $\mathbf{M}_f$, LRMT-FBT keeps

Table 9. Runtime and output level breakdown for LRMT-FBT. Time and output level are reported in separate columns. Non-total rows report time in milliseconds, while Total rows report time in seconds. A dash indicates that no output level was reported in the log.

Module	Step	$P = 2^{12}$, $p = 2^3$		$P = 2^{14}$, $p = 2^4$		$P = 2^{16}$, $p = 2^4$		$P = 2^{18}$, $p = 2^4$		$P = 2^{20}$, $p = 2^5$	
		Time	**Level**	**Time**	**Level**	**Time**	**Level**	**Time**	**Level**	**Time**	**Level**
Input conversion	RLWE→CKKS (ms)	26.65	0	30.75	0	34.88	0	34.20	0	82.19	0
FDD	ModRaise (ms)	240.79	23	247.06	24	285.11	24	322.01	25	574.72	26
	CoeffToSlot (ms)	32,058.75	18	47,867.60	20	53,071.48	20	61,330.75	21	76,481.85	21
	ComputeExp (ms)	54,258.98	10	64,807.11	12	64,867.82	12	80,029.08	13	137,378.31	13
	Kronecker delta (ms)	15,917.39	6	39,774.22	7	44,837.70	7	54,575.17	8	353,438.19	7
	SlotToCoeff (ms)	4,471.54	0	12,570.34	0	13,015.73	0	26,526.60	0	75,725.52	0
	Total (s)	**107.48**	–	**165.79**	–	**176.64**	–	**223.48**	–	**645.05**	–
Cleaning	Cleaning (ms)	-	-	20,731.00	5	25,888.43	5	28,633.27	6	112,580.96	5
Multiplexer tree	Kronecker product (ms)	8,794.00	5	18,333.81	4	29,462.07	4	136,891.99	4	259,220.92	4
	LUT evaluation (ms)	2,124.98	3	3,485.11	2	5,249.72	2	13,015.62	2	42,622.93	2
	Total (s)	**11.02**	–	**21.82**	–	**34.72**	–	**150.48**	–	**304.96**	–
Output conversion	SlotToCoeff (ms)	1,348.06	0	3,399.72	0	3,390.68	0	3,329.49	0	13,651.73	0
	CKKS→RLWE (ms)	35.23	–	34.41	–	34.64	–	34.82	–	69.36	–
Overall	**Total (s)**	**119.91**	–	**211.80**	–	**240.71**	–	**405.98**	–	**1,076.39**	–

Table 10. Multi-threaded latency and amortized time for LRMT-FBT on ReLU ($r = 2$). The amortized time is the latency divided by the N packed values.

	$P = 2^{12}$	$P = 2^{14}$	$P = 2^{16}$	$P = 2^{18}$	$P = 2^{20}$
Latency (s)	42.39	70.67	78.12	109.34	275.03
Amortized time (ms)	0.65	1.08	1.19	1.67	2.10

only the rank-r factors $\{(\sigma_k, \mathbf{u}_k, \mathbf{v}_k)\}_{k=1}^{r}$, that is, $r(2\sqrt{P}+1)$ scalars. At $P = 2^{20}$ and $r = 2$, this reduces the stored values from 2^{20} to 4,098, or 32.02 KiB. The bootstrapping keys, which are shared by both schemes, occupy 39.66 GiB in total, comprising 216.01 MiB of multiplication keys and 39.45 GiB of rotation keys.